

Software Dependability

Lab Session: Docker, Docker Compose, and Kubernetes

What are we going to do Today?

Analyzing some tools for containerization and orchestration

(Docker, Docker Compose, Podman, Docker Swarm, and Kubernetes)

Dockerize a Maven Project

<https://www.educative.io/answers/how-do-you-dockerize-a-maven-project>

The second part of your project assignment consists of dockerizing your project and describing the Dockerfile in your report.

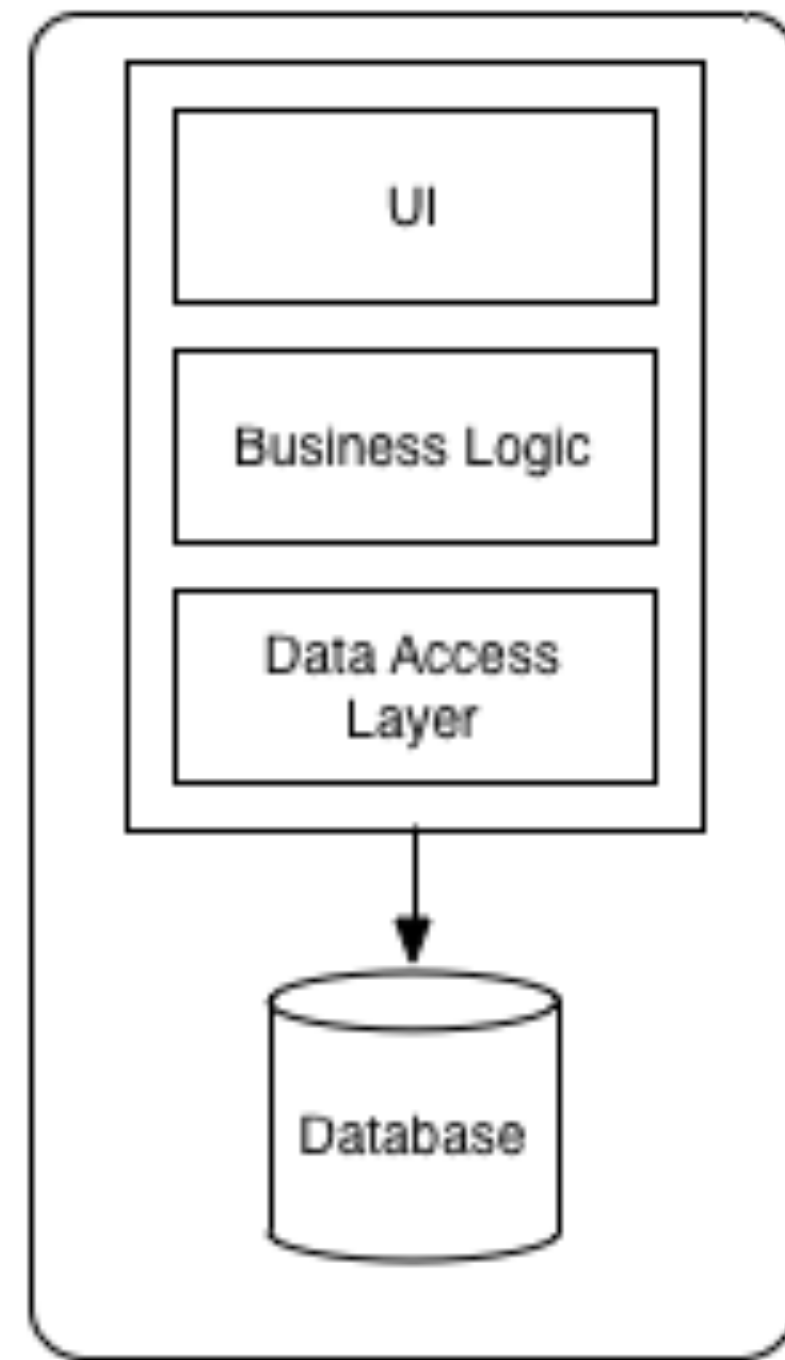
Play with Docker basics

<https://docs.docker.com/get-started/workshop/>

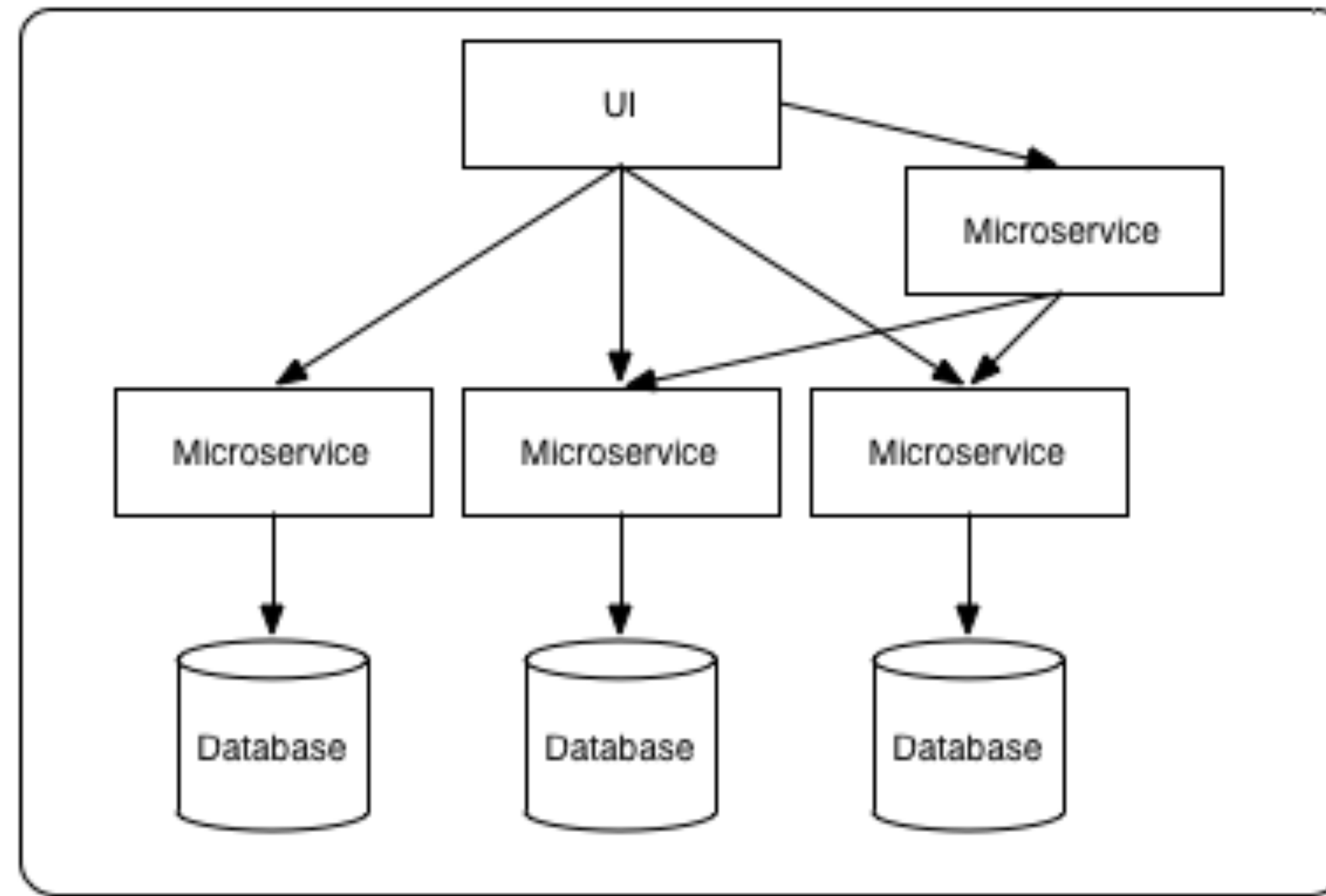
Get started with Kubernetes

<https://minikube.sigs.k8s.io/docs/start/>
https://minikube.sigs.k8s.io/docs/tutorials/kubernetes_101/

Continuous Architecting & Microservices



Monolithic Architecture

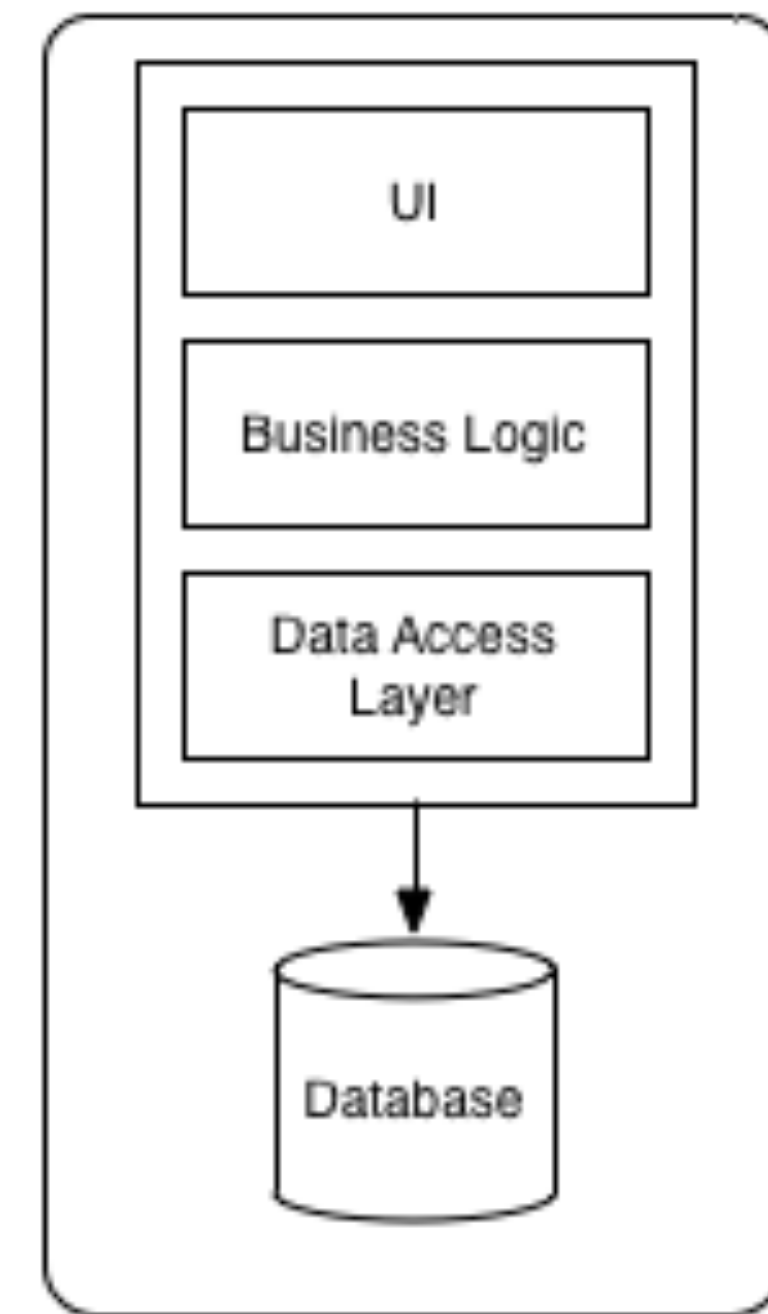


Microservices Architecture

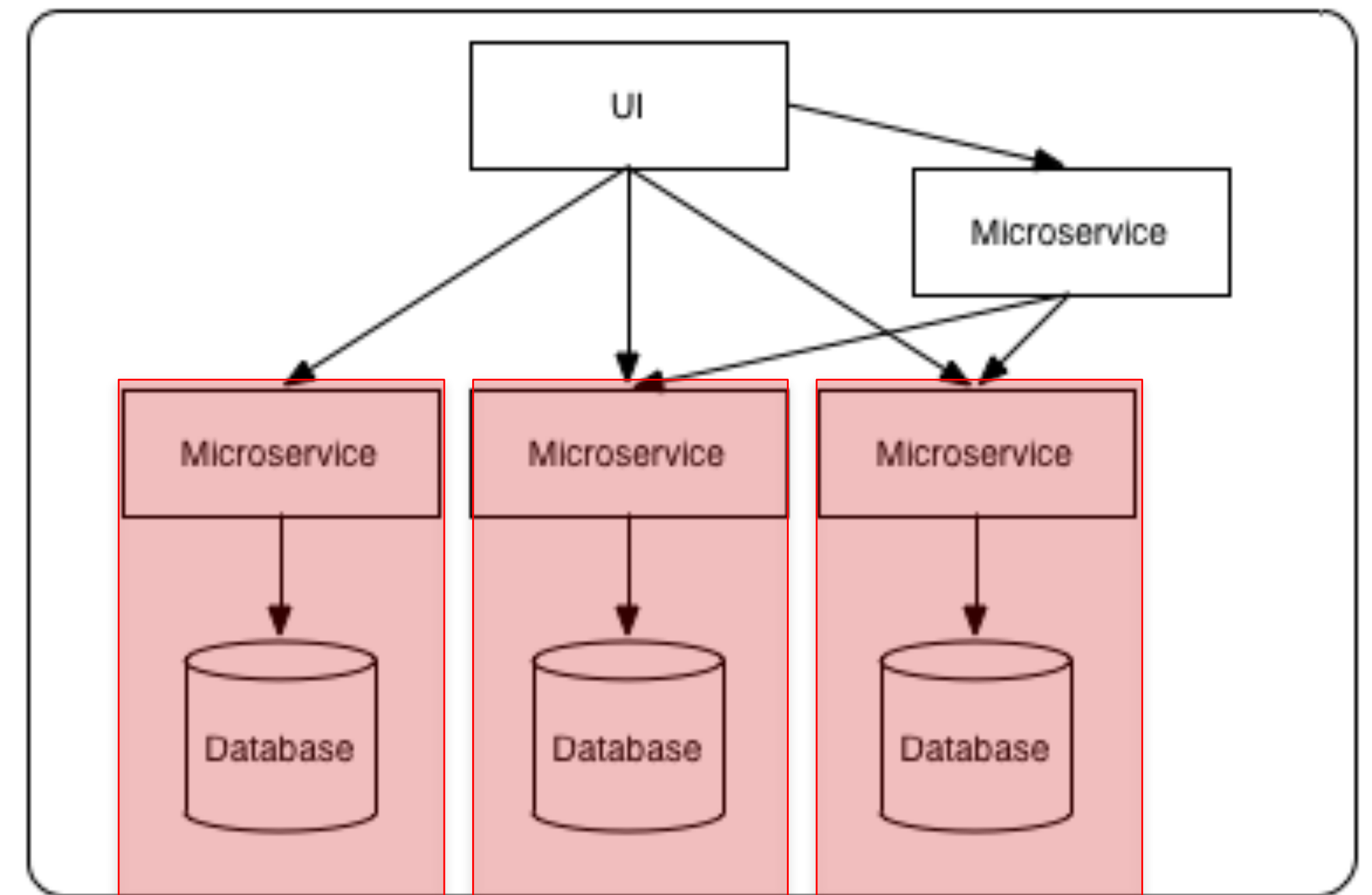
Microservices

Architectural principles

- Statelessness;
- Available in multiple copies;
- No data exchange between peers;
- Each microservice has its own database;
- Data-Intensive Message Brokering;
- Message queues.



Monolithic Architecture



Microservices Architecture

Container!

Microservices Benefits

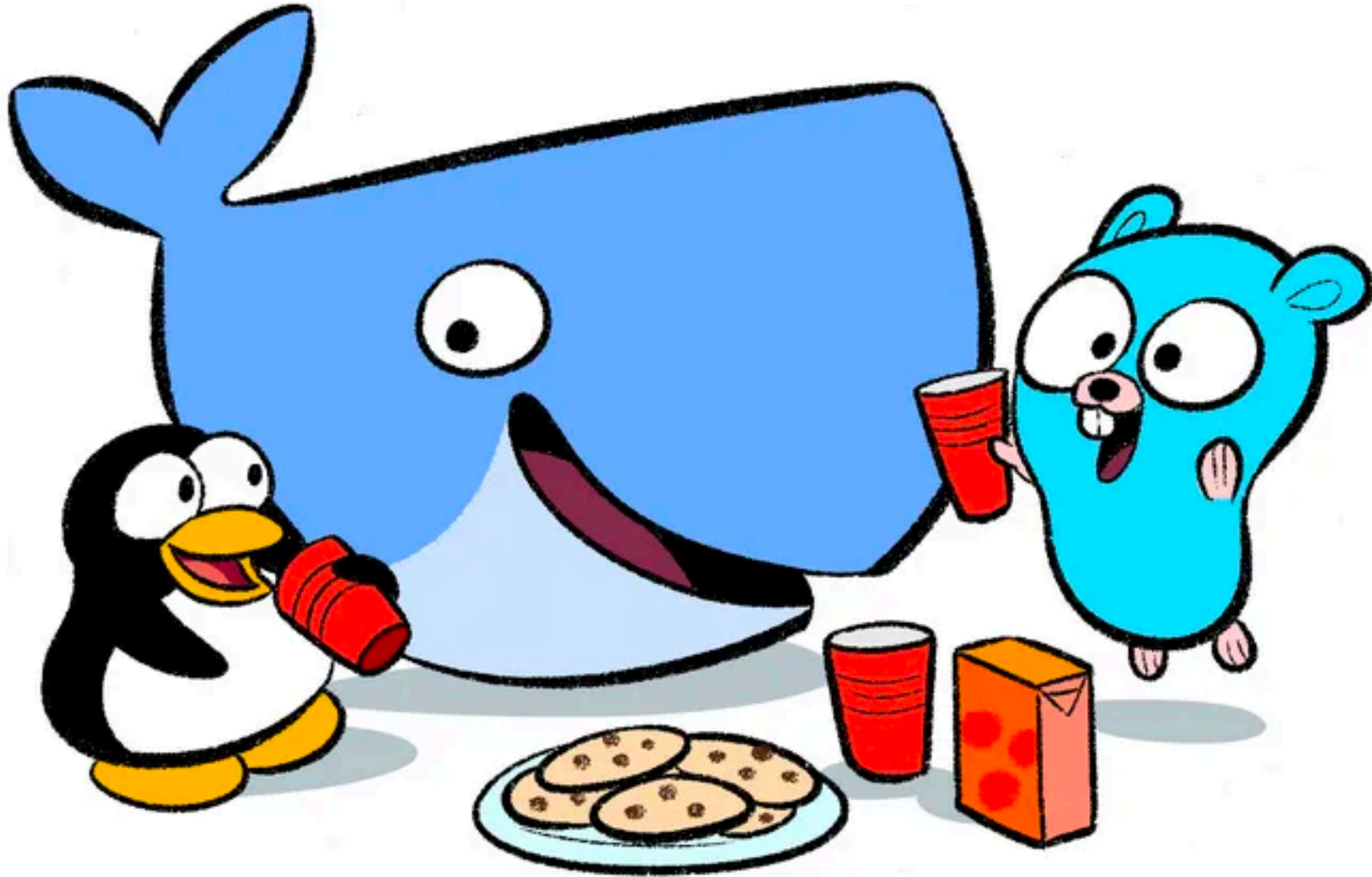
Benefit	Description
Containerization	Helps promote technologies like Docker and Cloud development
Easier Deployments	Much easier to push new features of applications with less reliance on the entire application to be redeployed
Resilient	Services are self-reliant and do not depend on other services reducing the bottlenecks and failure points.
Scalability	Self-contained servers can be deployed to spread the requirements for the application
Agile development	MSA was designed to encourage short cycle times and faster development time for updates and creation.
More Flexibility	Using APIs to communicate means that teams have the ability to use the proper language or technology to best suit their needs.

Containerization

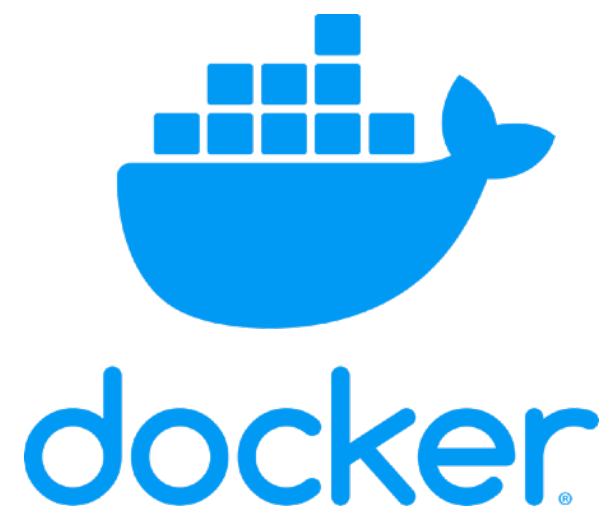
"By containerization we mean the **grouping of software code and all its necessary components**, such as libraries, frameworks and other dependencies, so that they are **isolated** in their own space that we call **container**."

"By grouping all the elements of an application into a container that can be **moved between platforms and infrastructures**, the application can be used where needed."

Docker



Docker

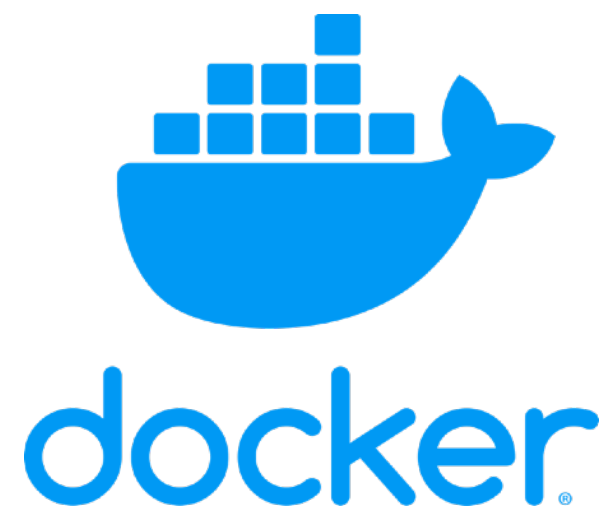


"Docker is an open source project that automates the delivery of applications within software containers, providing an additional layer of abstraction and automation of virtualization at the operating system level."

Docker provides a uniform wrapping around a software package according to the philosophy "Create, ship and run any app, anywhere"

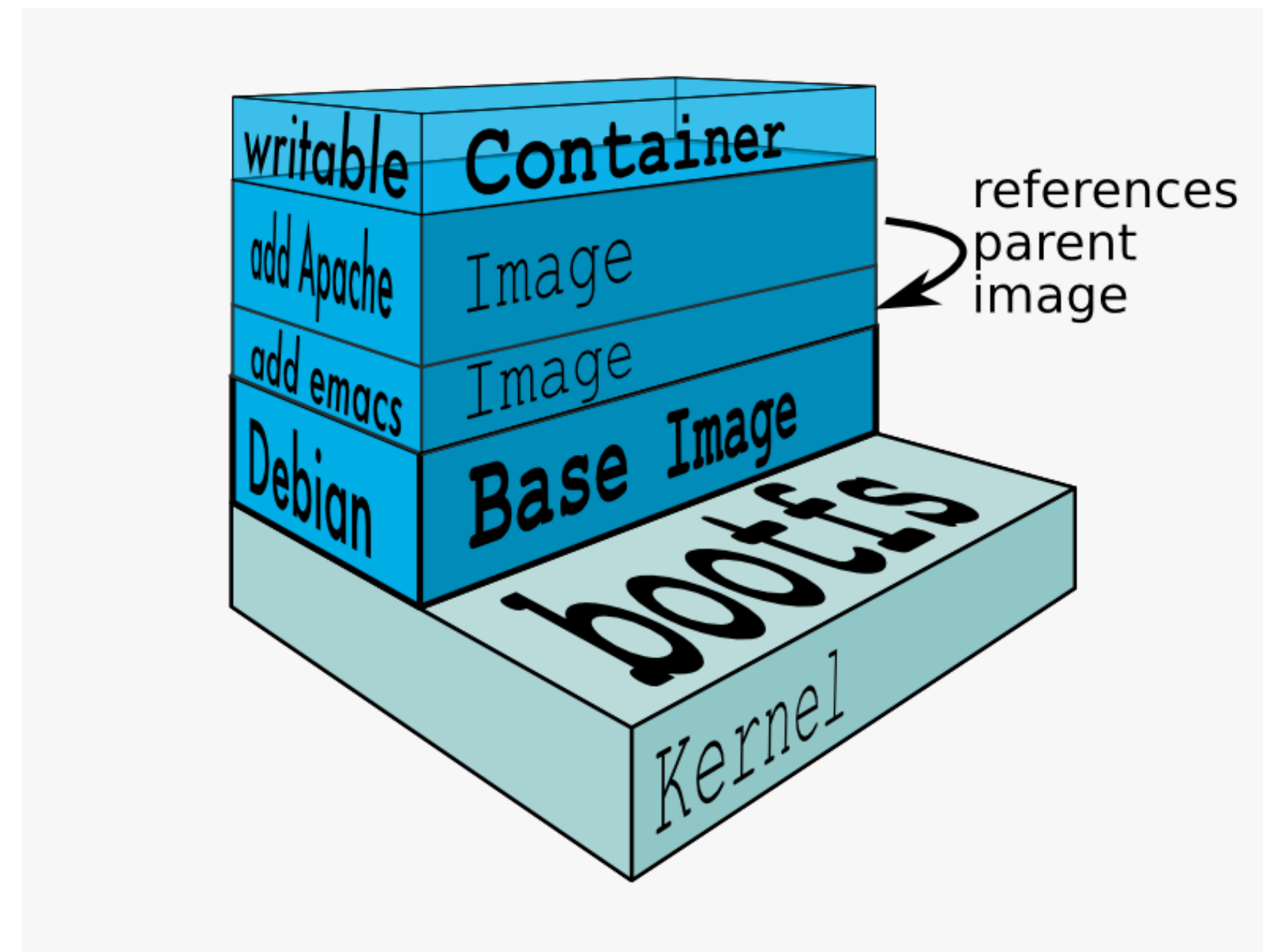
Like shipping containers: the container is always the same, regardless of the content and can be transported by trucks, cranes, ships, ...

Docker: Technologies



Docker integrates several technologies

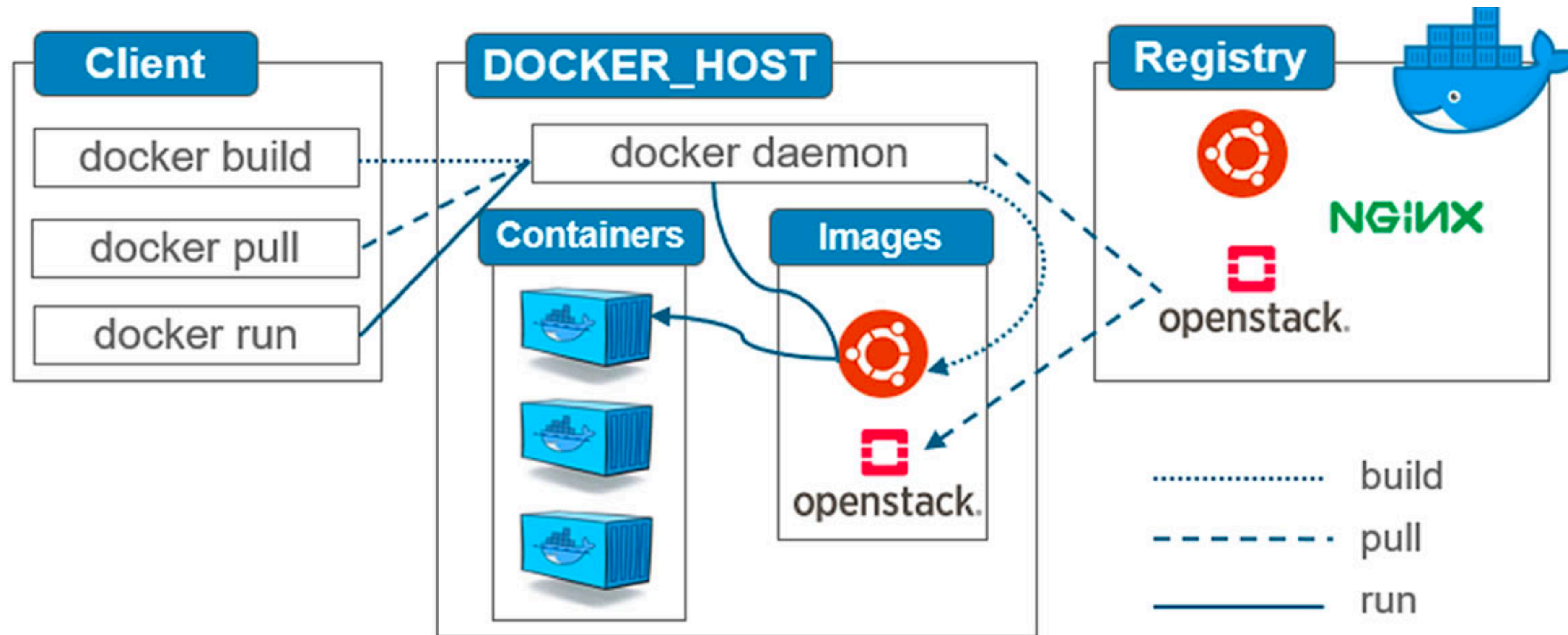
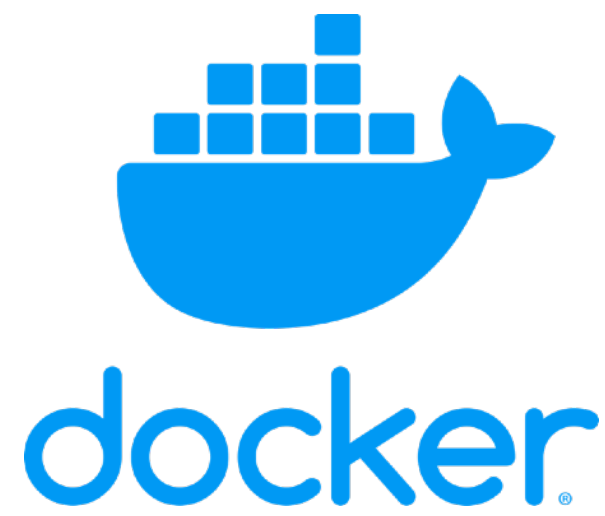
- libvirt: virtualization platform
- LXC (Linux Containers): isolated instances of multiple linux systems (containers) on a single node
- Layered file system



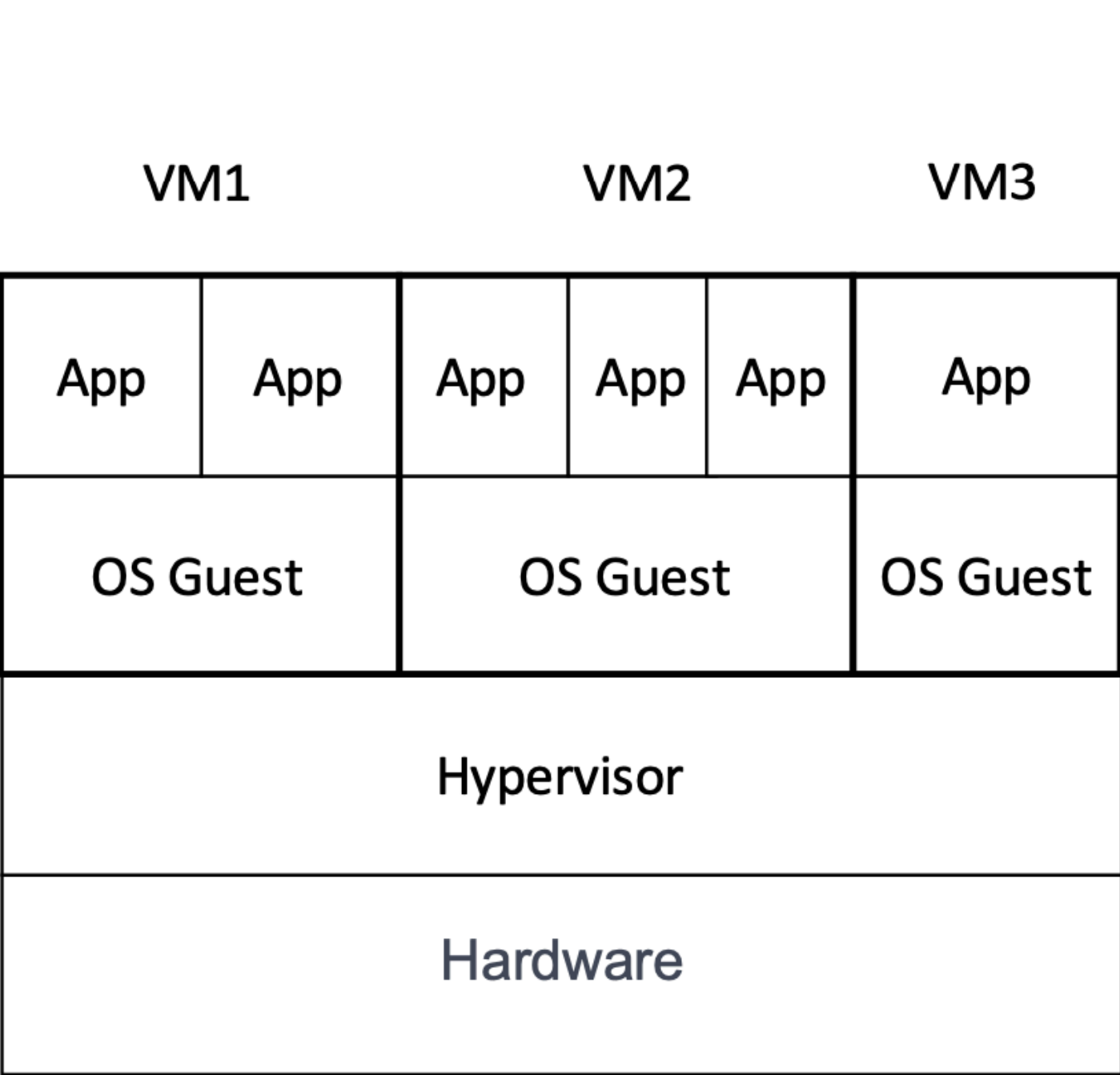
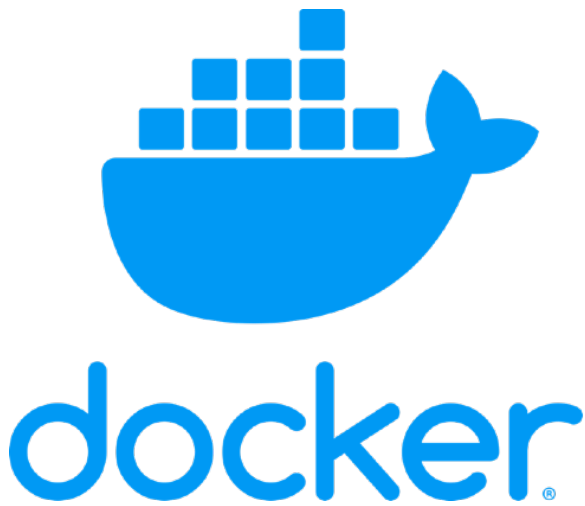
Platforms on which Docker can be installed

- Various Linux distributions (e.g., Ubuntu, Fedora, RHEL, Centos, openSUSE)
- Cloud (e.g., Amazon EC2, Google Compute Engine, Rackspace)
- 10-2014: Microsoft announces Docker integration into Windows Server

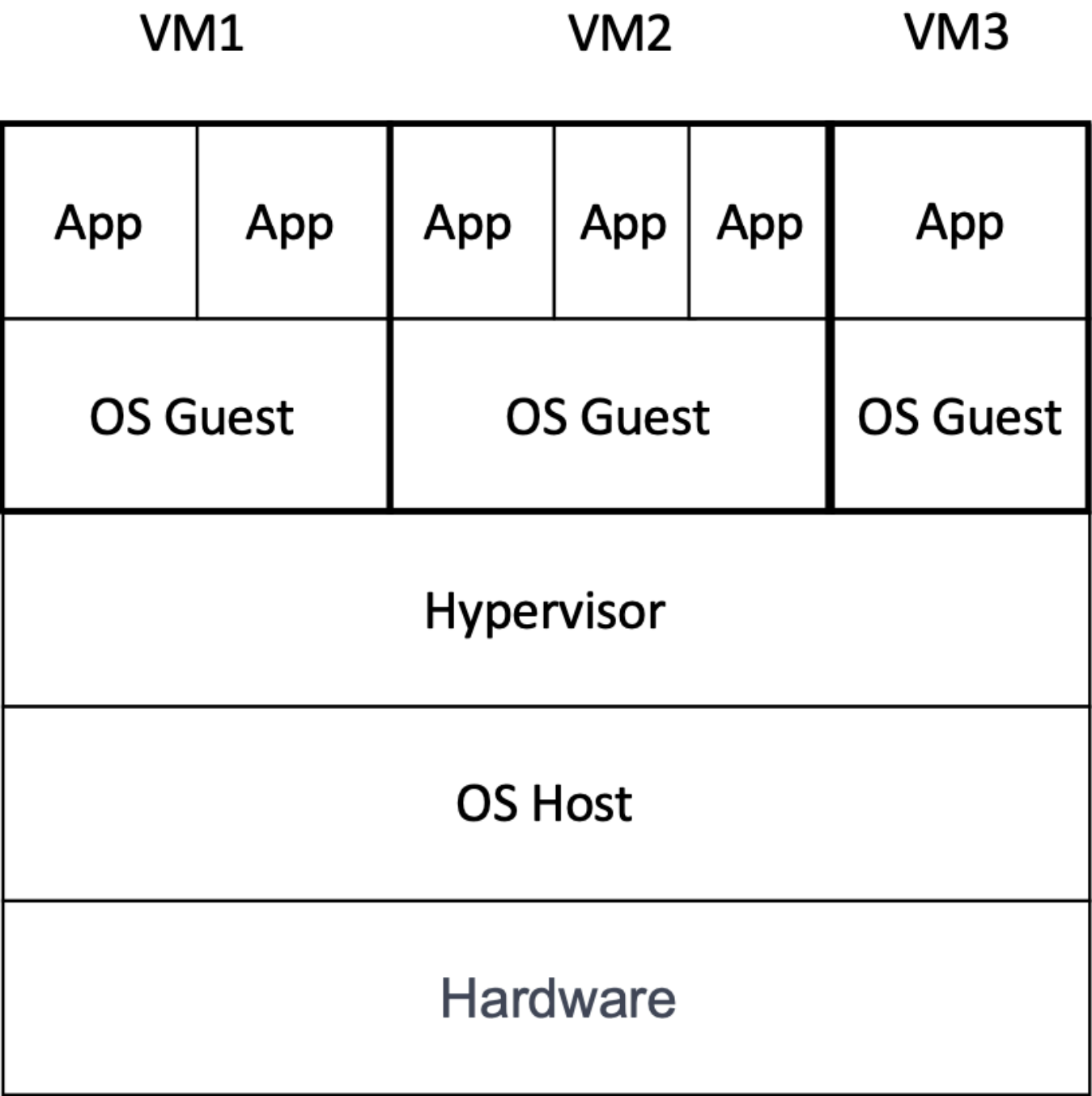
Docker: Architecture



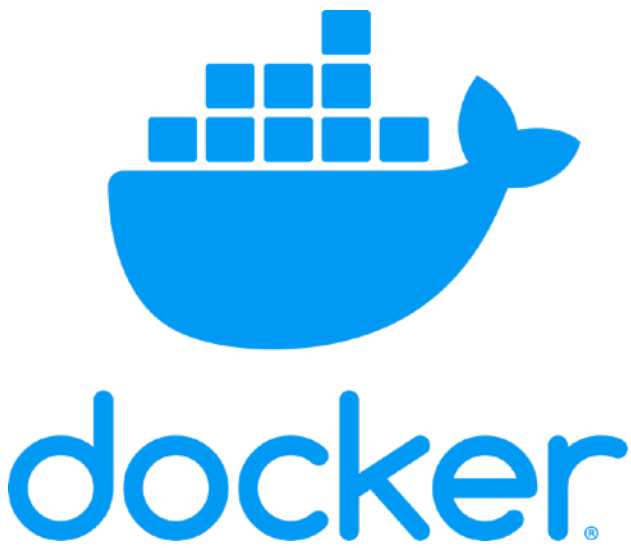
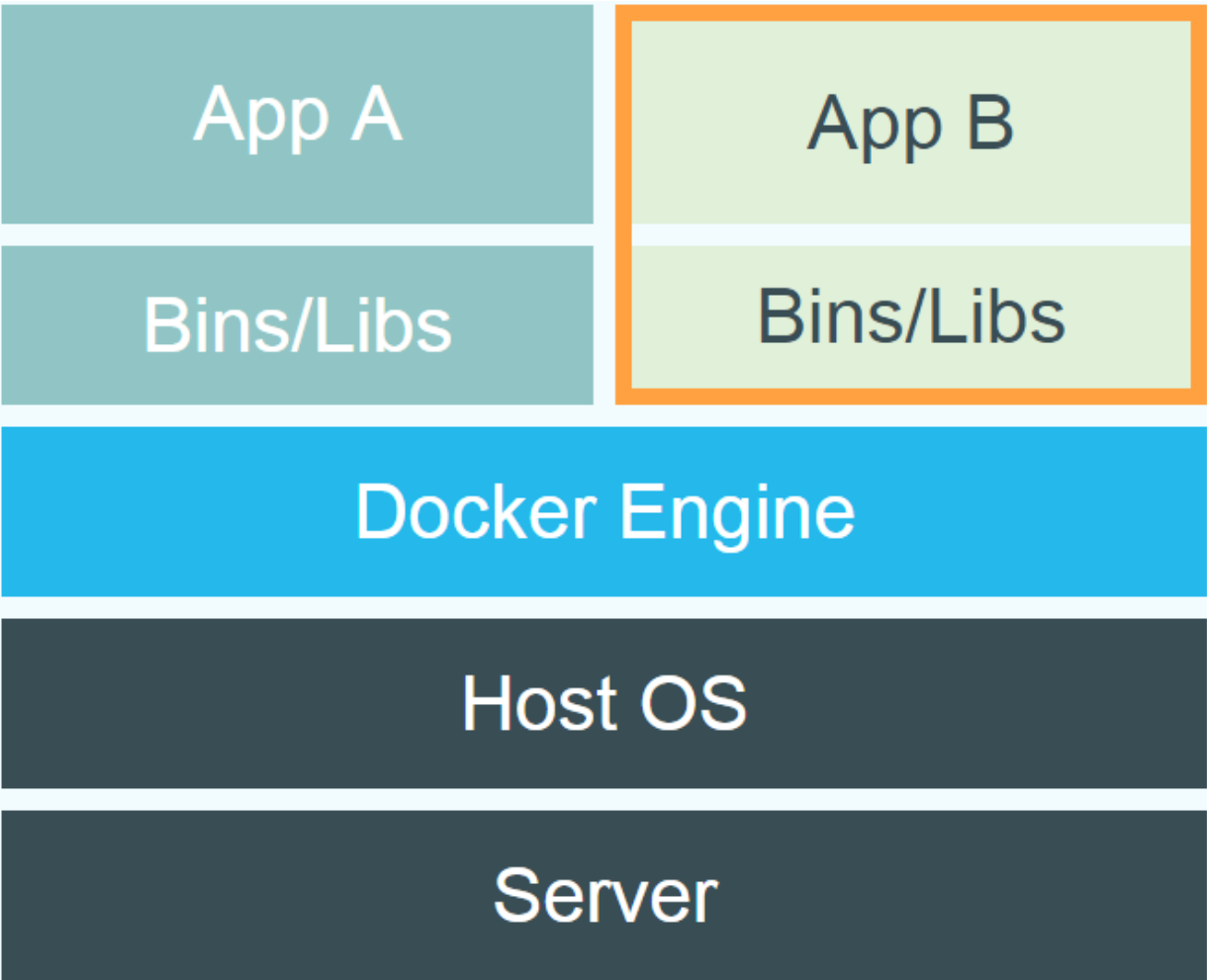
Docker vs Virtual Machines



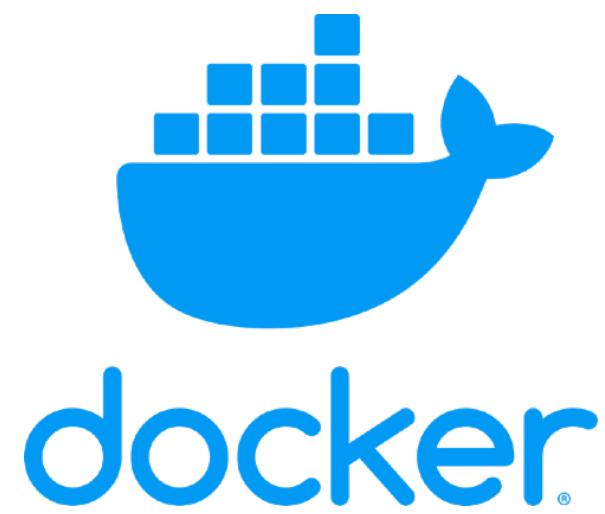
A bare-metal hypervisor



A hosted hypervisor



Docker: Pros and Cons



Pro

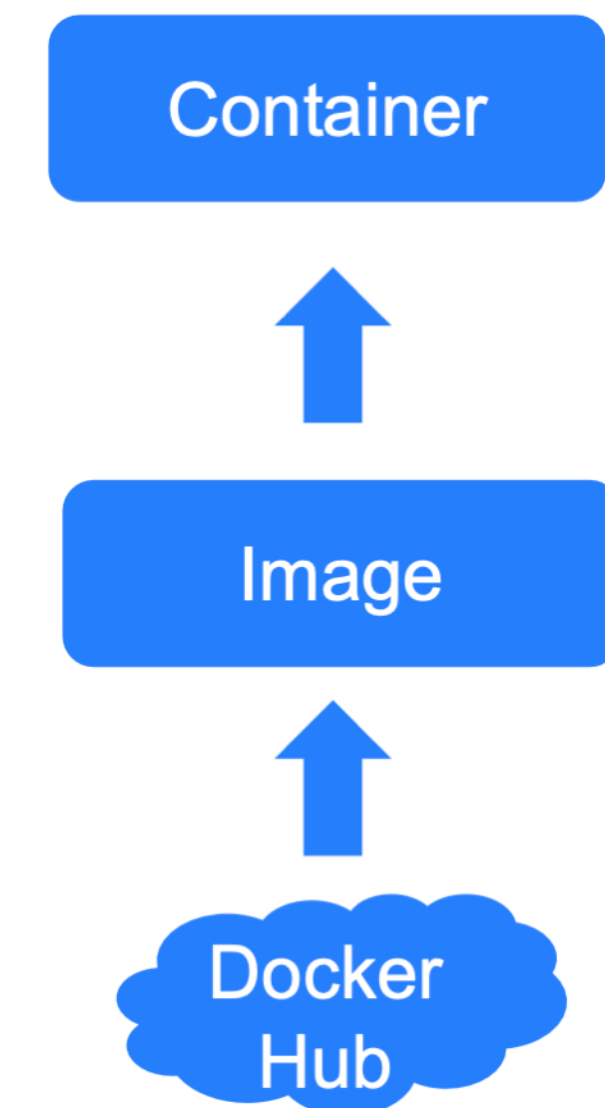
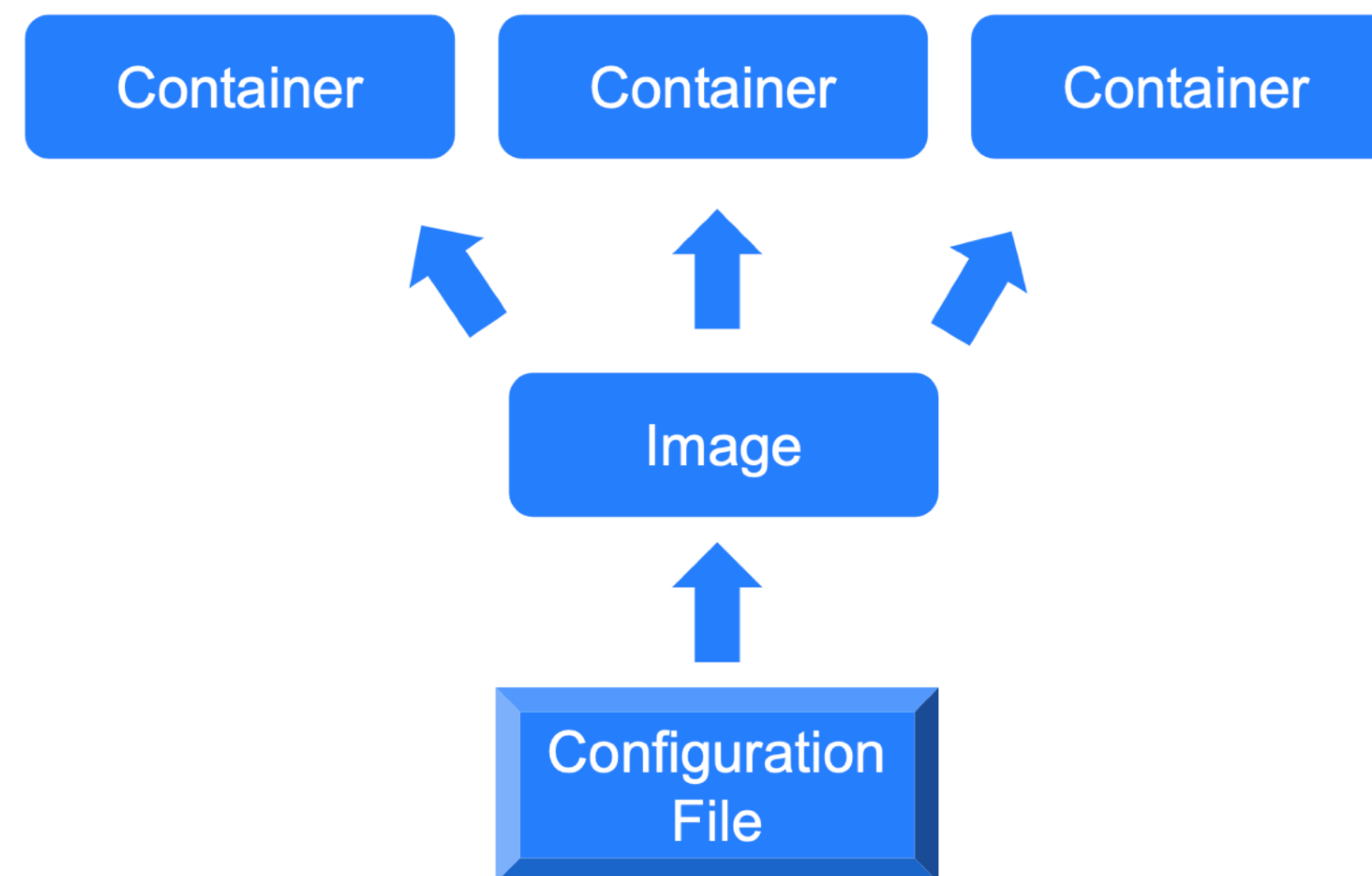
- The system is very light and the overhead is minimum
- Containers are very small and take seconds to be booted
- Open source
- Multi platform

Cons

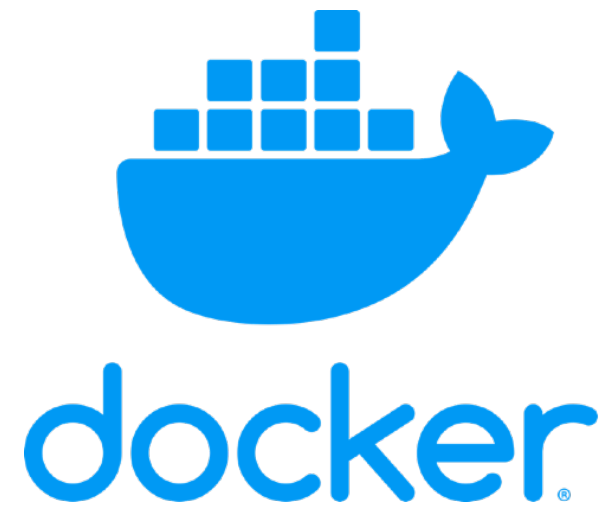
- There is no complete separation: security problems
- Can emulate only OS of the host type

The configuration file is called Dockerfile. It permits to:

- Define base image
- Set up environment (download extra packages, ...)
- Configure host (map ports..)
- Execute commands
- Copy files inside the container
- Start/Stop the container



Docker 101



Basic commands

- `docker run ubuntu echo Hello World` # Create and run a docker container from the ubuntu image
then it executes echo Hello world
- `docker images -a` # Lists all images on the host
- `docker ps -a` # Lists all containers on the host

To define an image you need a "Dockerfile"

- The images are created automatically starting from a script called Dockerfile.
- The Dockerfile can be placed in a version control system such as Git, along with its dependencies.
- Docker Hub can build images automatically starting from docker files on GitHub.

Dockerfile Example

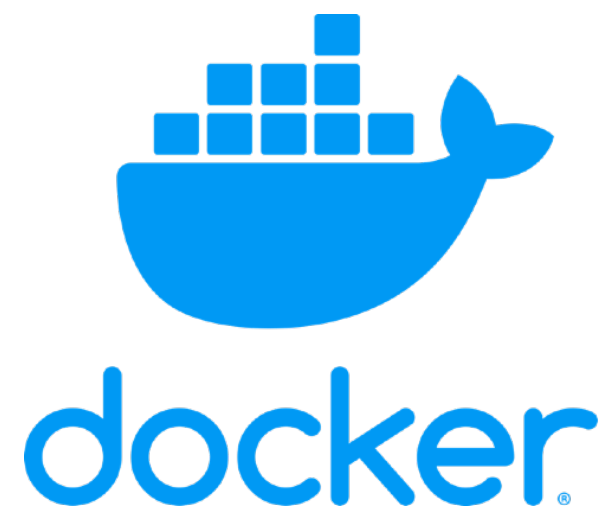
```
FROM ubuntu
ENV DOCK_MESSAGE Hello My World
ADD dir /files
CMD ["bash", "someScript"]
```

<https://hub.docker.com/>

The image is built with `docker build [DockerFileDir]`

Information on the newly built image is obtained through `docker inspect [imageId]`

Docker: Use Cases

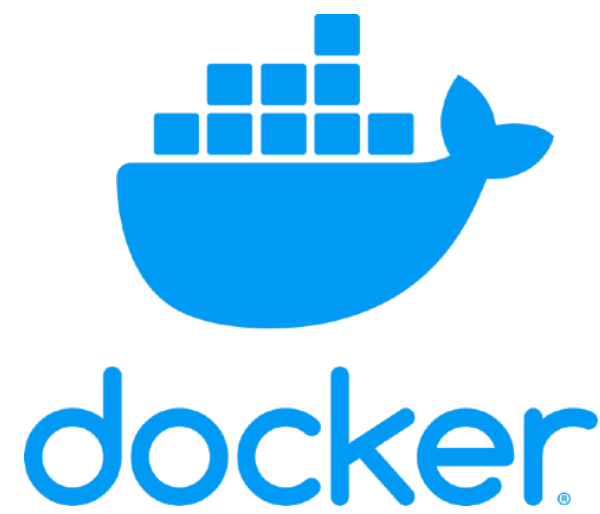


- Development environments
- Environments for integration testing
- Quick software evaluation
- Microservices
- Multi-Tenancy (when we want to test and put into production a software in different environments)
- A unified execution environment: development -> testing -> production (local, VM, cloud, ...)

Docker: Extra Maintenance

- Maintenance of IaC
- Maintenance of containerization technology
- Infrastructure backend maintenance (if owned)

Docker: Let's see basics



<https://docs.docker.com/get-started/>

<https://docs.docker.com/get-started/workshop/>

Docker: Sample Containers in other Languages

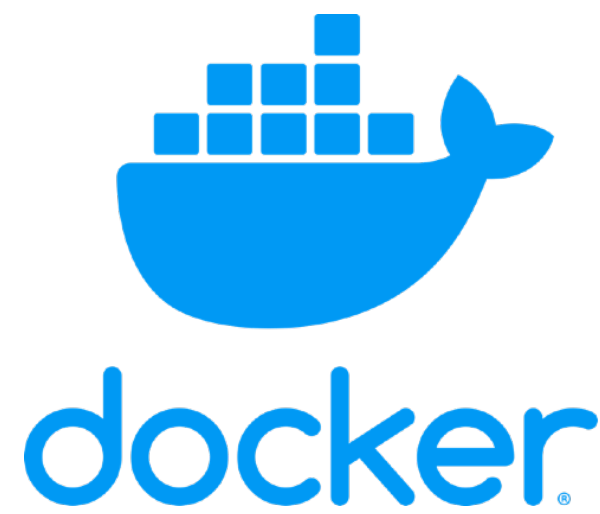
Select some language-specific containers, analyze them, and run them with Docker!

<https://docs.docker.com/reference/samples/>

Docker: Common Mistakes

- Running `apt-get update/upgrade`
- Using `ADD` instead of `COPY`
- Adding your entire application directory in one line
- Using `:latest`
- Using external services during the build
- Multiple `FROM` statements
- Multiple services running in the same container

How to Handle Multiple Docker Containers?



Docker Compose is a tool for defining and running multi-container Docker applications.

With Docker Compose, you use a **YAML** file to configure your application's services.

Then, with a single command, you create and start **all the services from your configuration**.

However, Compose configures all containers on the same host.

What is YAML?

It is a human readable markup language
(Yet Another Markup Language) dialect of JSON.

What can you do with Docker Compose?

- Start, stop, and rebuild services.
- View the status of running services.
- Stream the log output of running services.
- Run a one-off command on a service.

Yet Another Markup Language (YAML)

YAML is a super set of JSON. You need to know two structure for config files.

Maps

```
apiVersion: v1
kind: Pod
  metadata:
    name: rss-site
    labels:app: web
```

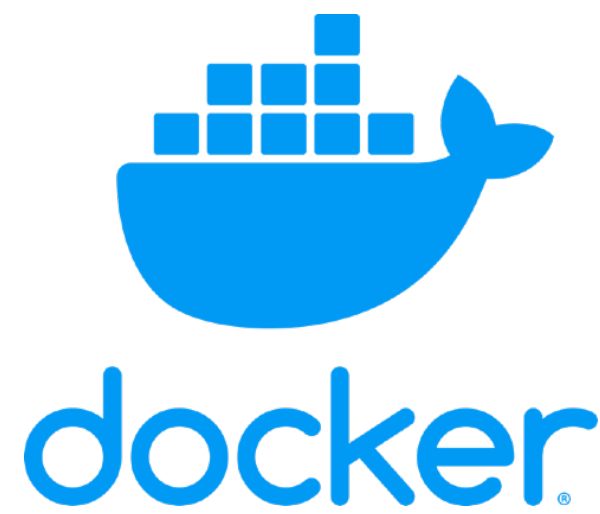
Lists

```
args:
- sleep
- "1000"
- message
```

Benefits

- Convenience: no longer need to specify all parameters to the command line.
- Maintenance: can be added to source control, to track changes.
- Flexibility: allows to create much more complex structures.

Docker Compose in Three Steps



1. Define your app's environment with a **Dockerfile** to reproduce it anywhere.
2. Define the services that make up your app in **docker-compose.yml** so they can be run together in an isolated environment.
3. Run **docker compose up**, and the Docker compose command starts and runs your entire app.

Key Features of Docker Compose

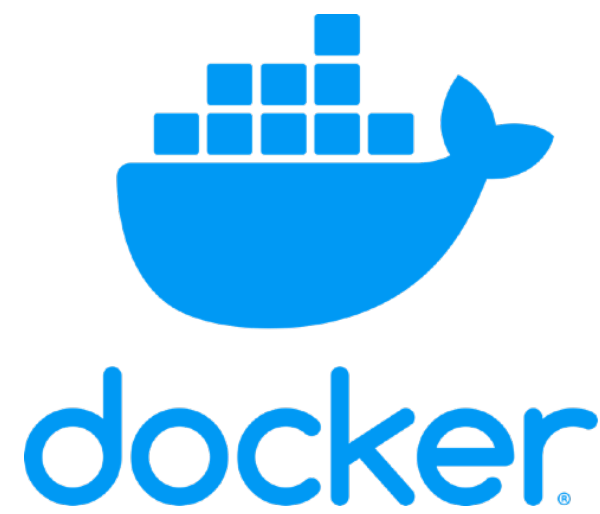
- Have multiple isolated environments on a **single host**.
- Preserves volume data when containers are created.
- Only recreate containers that have changed.
- Supports variables and moving a composition between environments.

On a dev host, to create multiple copies of a single environment, such as when you want to run a stable copy for each feature branch of a project.

On a CI server, to set the project name to a unique build number to keep builds from interfering with each other.

On a shared host or dev host, to prevent different projects that may use the same service names from interfering.

Docker Compose: Let's see basics



https://docs.docker.com/get-started/08_using_compose/

Docker Compose: Sample Applications

Select some sample applications, analyze them, and run them with Docker Compose!

<https://github.com/docker/awesome-compose>

Docker Compose: another Tutorial

<https://docs.docker.com/compose/gettingstarted/>



podman

Podman



Podman (the POD manager) is an open source tool for developing, managing, and running containers on your Linux systems. Originally developed by Red Hat, Podman manages the entire container ecosystem using the libpod library.

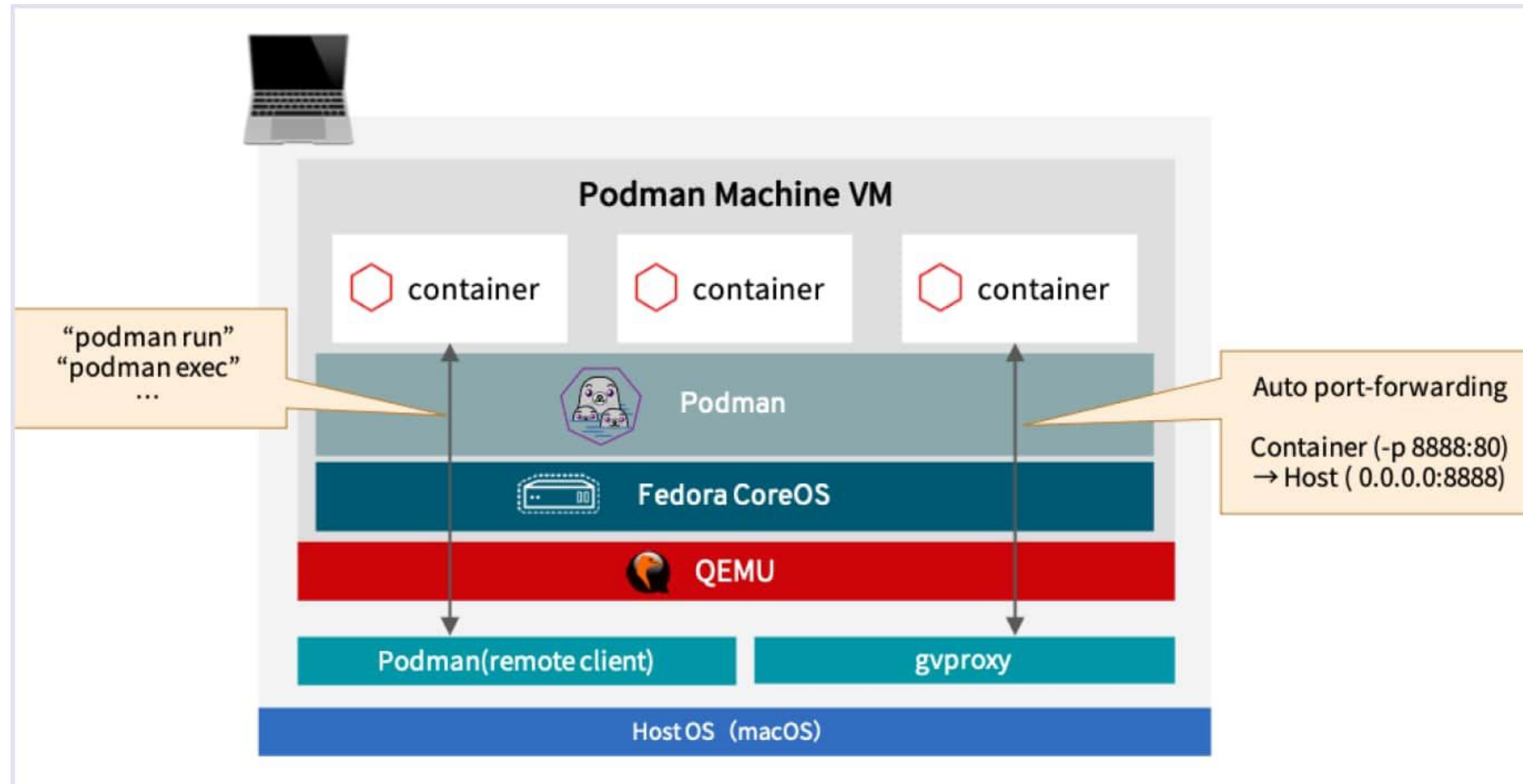
Podman's daemonless and inclusive architecture makes it a more secure and accessible option for container management, and allows developers to customize their container environments to best suit their needs.

Podman: Technologies



PodMan integrates several technologies

- QEMU: virtualization platform
- LXC (Linux Containers): isolated instances of multiple linux systems (containers) on a single node
- Layered file system
- Gvproxy: proxy allowing for auto port-forwarding



Podman: Pros and Cons

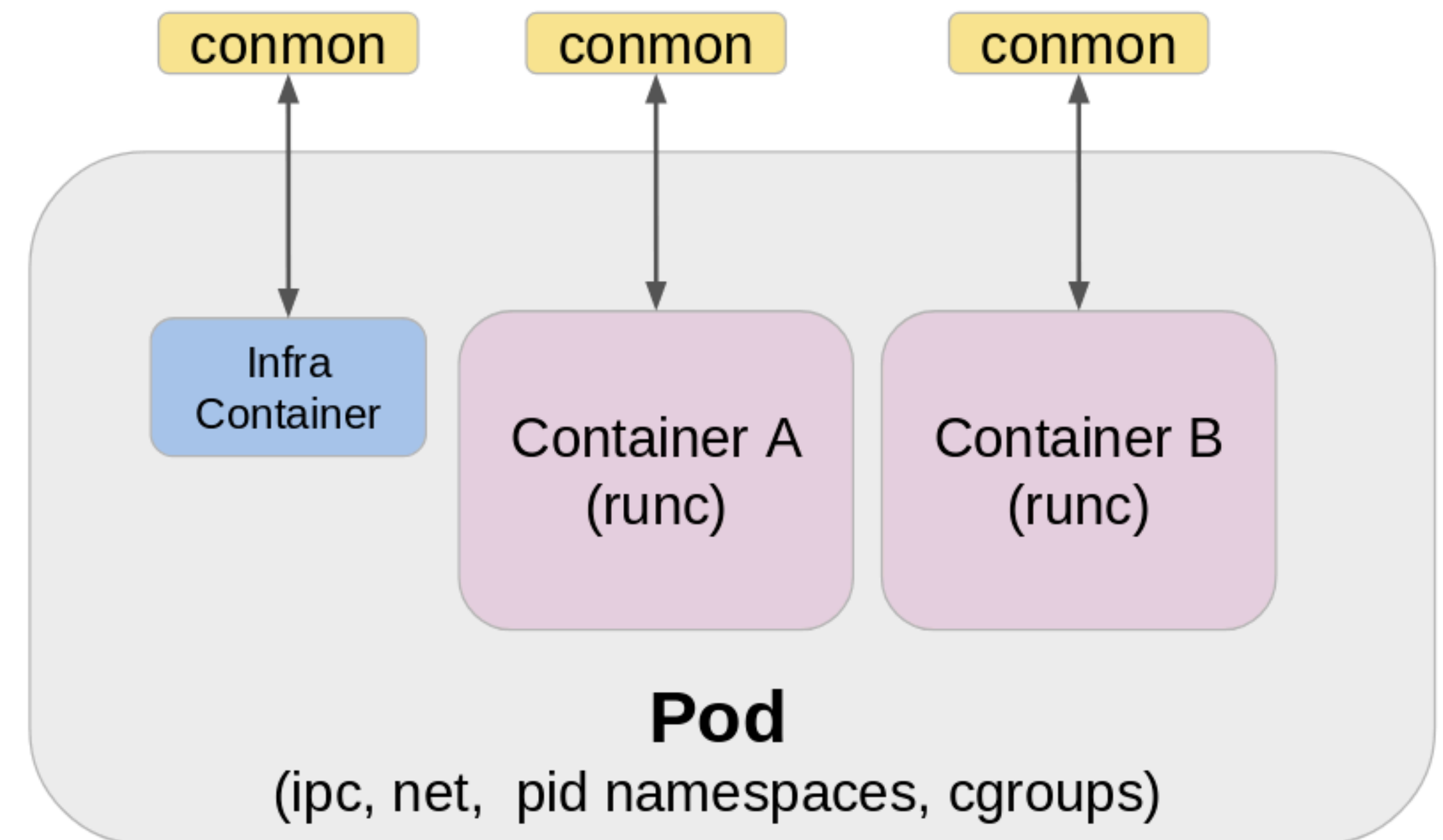


Pro

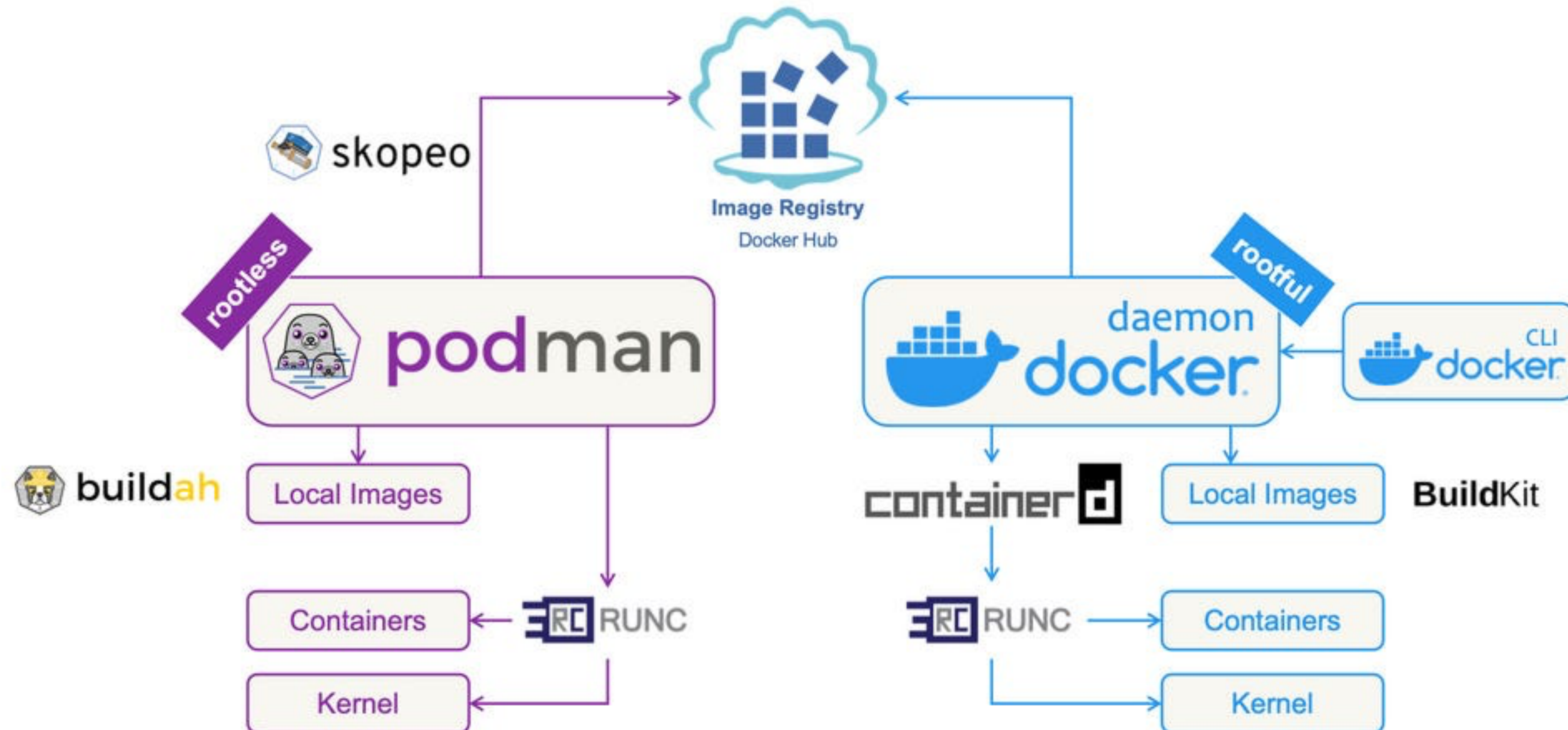
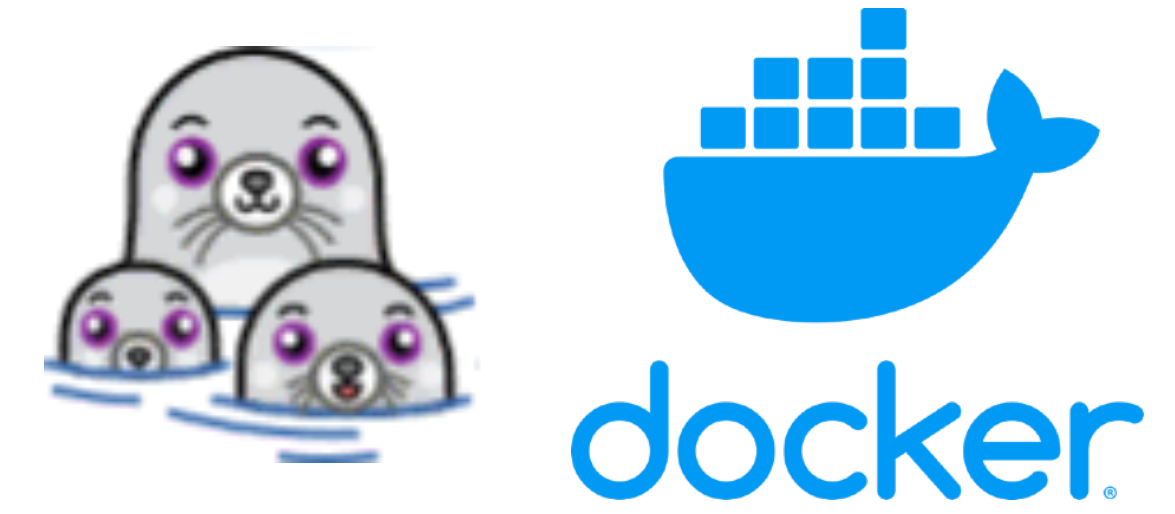
- Security: it uses a rootless architecture, which means that it runs containers as non-root users
- Compatibility: its images are compatible with Docker
- Performance: it has a minimal footprint and does not require a daemon to run
- Flexibility: it is a flexible tool that can run both containers and pod
- Open source
- Multi platform

Cons

- Complexity: it has a steeper learning curve than Docker
- Community support: it has relatively small community support compared to Docker



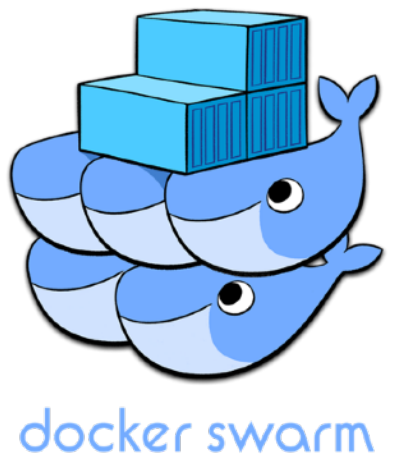
Podman vs Docker



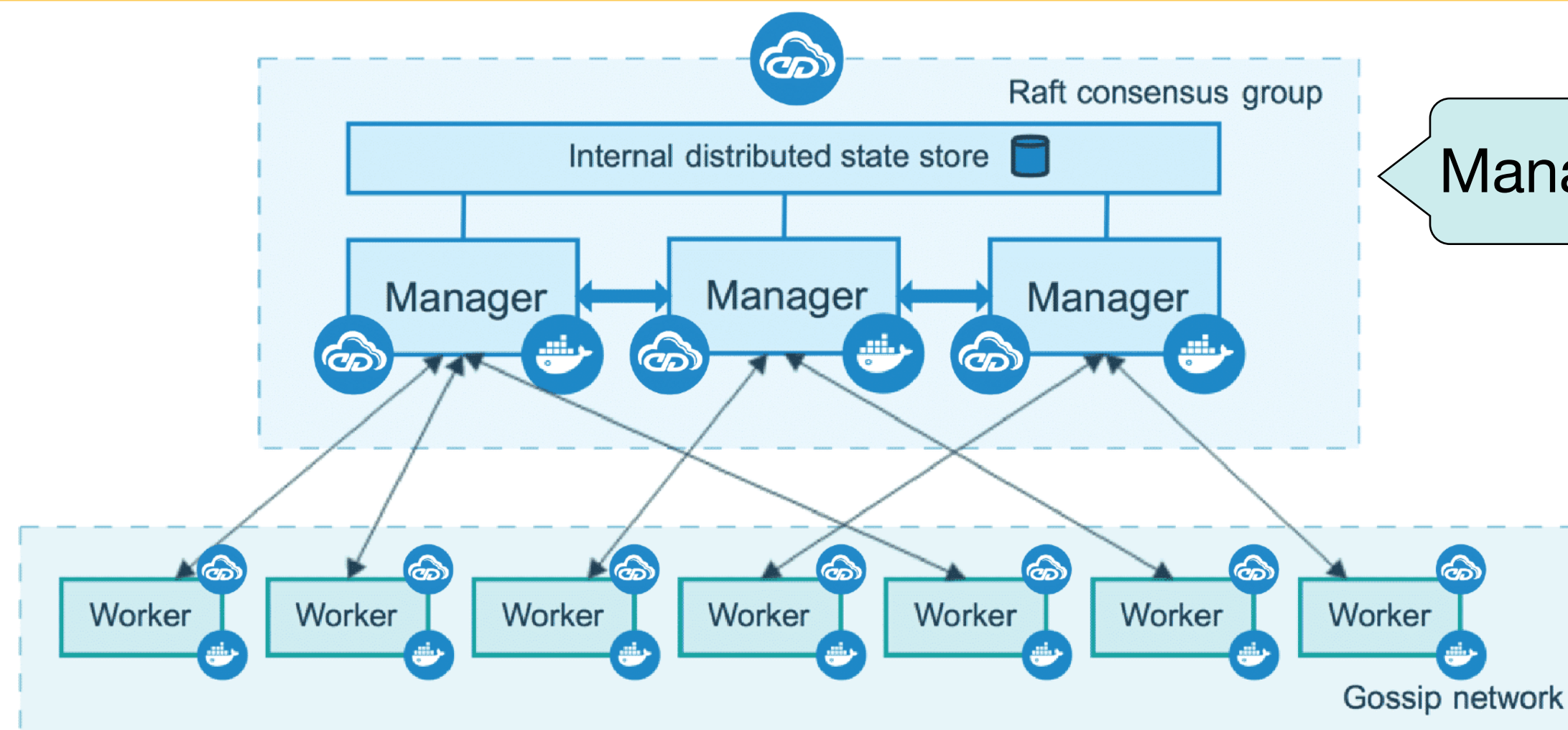
What about Multiple Hosts? Let the Orchestration Begin



Docker Swarm



Docker Swarm is a **container orchestrator** running the Docker application. It joins together docker containers in a cluster. A swarm manager controls the activities of the cluster, and machines that have joined the cluster are referred to as nodes.



Managers manage membership and delegation.

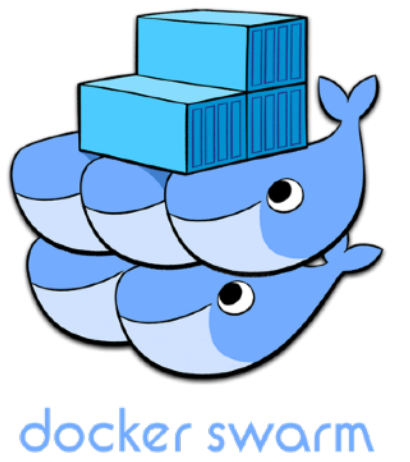
Workers run swarm services.

Docker Swarm manages Docker containers and not machines!

A swarm consists of **multiple Docker hosts**. A Docker host can be a manager, a worker, or perform both roles.

When you create a service, you define its optimal state (i.e., number of replicas, network and storage resources available to it, ports the service exposes to the outside world, and more). Docker Swarm works to maintain that desired state.

Docker Swarm: Components



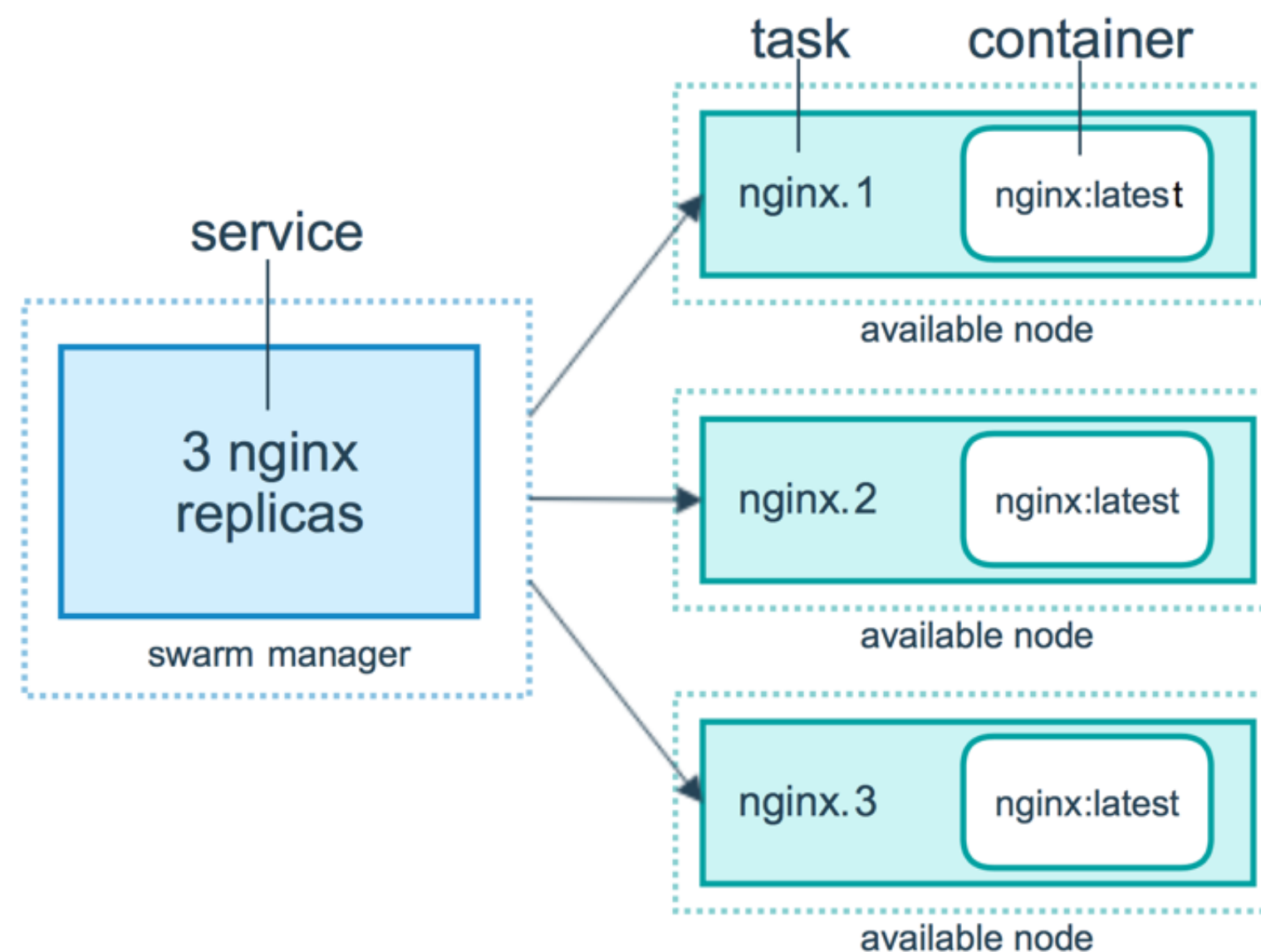
A Docker Swarm node is an instance of the Docker engine participating in the swarm.

To deploy an application to a Docker Swarm, you need to submit a **service definition** to a **swarm manager**, which dispatches units of work called **tasks** to **worker nodes**.

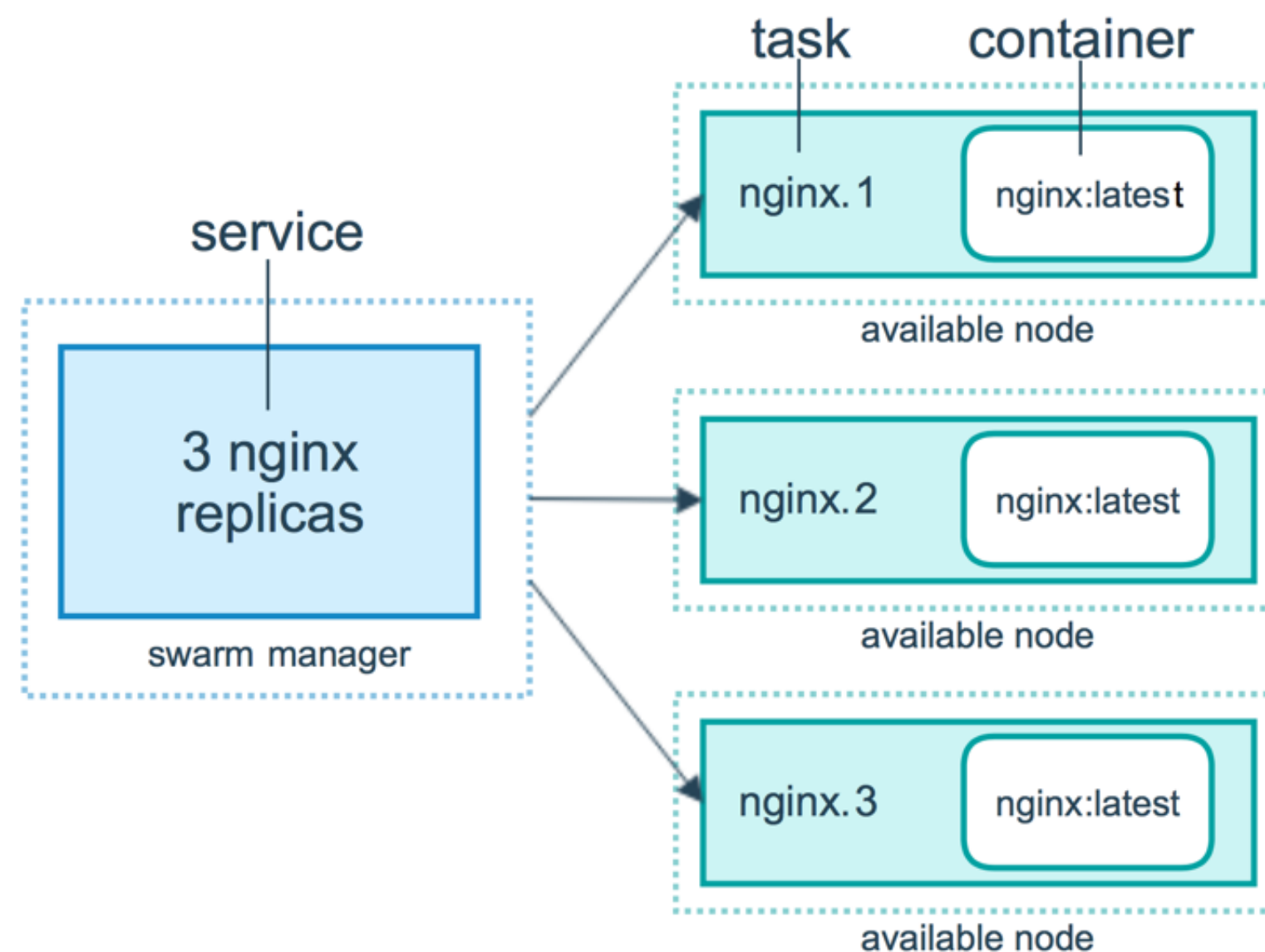
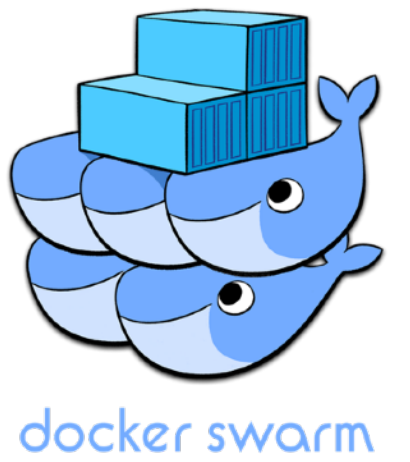
Swarm managers perform **orchestration** and **cluster management** to maintain the desired state of the swarm. A single swarm manager conducts the orchestration. They can also run services, but you can configure them to run only management activities.

Worker nodes receive and execute **tasks** dispatched from **manager nodes**.

An **agent** on each worker node reports on its assigned tasks to the swarm manager.



Docker Swarm: Components



A **service** defines the tasks to execute on the nodes.

When you create a service, you specify which container image to use and which commands to execute inside running containers.

For **replicated services**, the swarm manager distributes a specific number of replica tasks among the nodes based on the scale you set in the desired state.

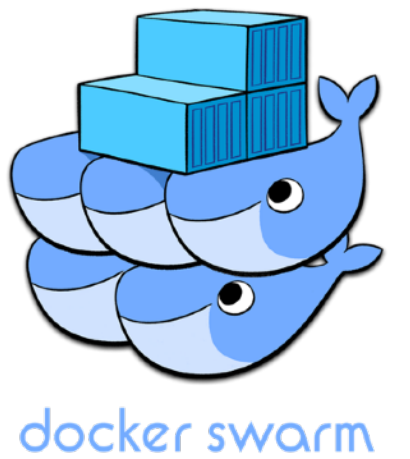
For **global services**, the swarm runs one task for the service on every available node in the cluster.

A **task** carries a Docker container and the commands to run inside the container: it is the atomic scheduling unit of the swarm.

Swarm managers assign tasks to workers based on the number of replicas set in the service scale.

Once a task is assigned, it cannot move to another node: it can run on the assigned node or fail.

Docker Swarm: Components



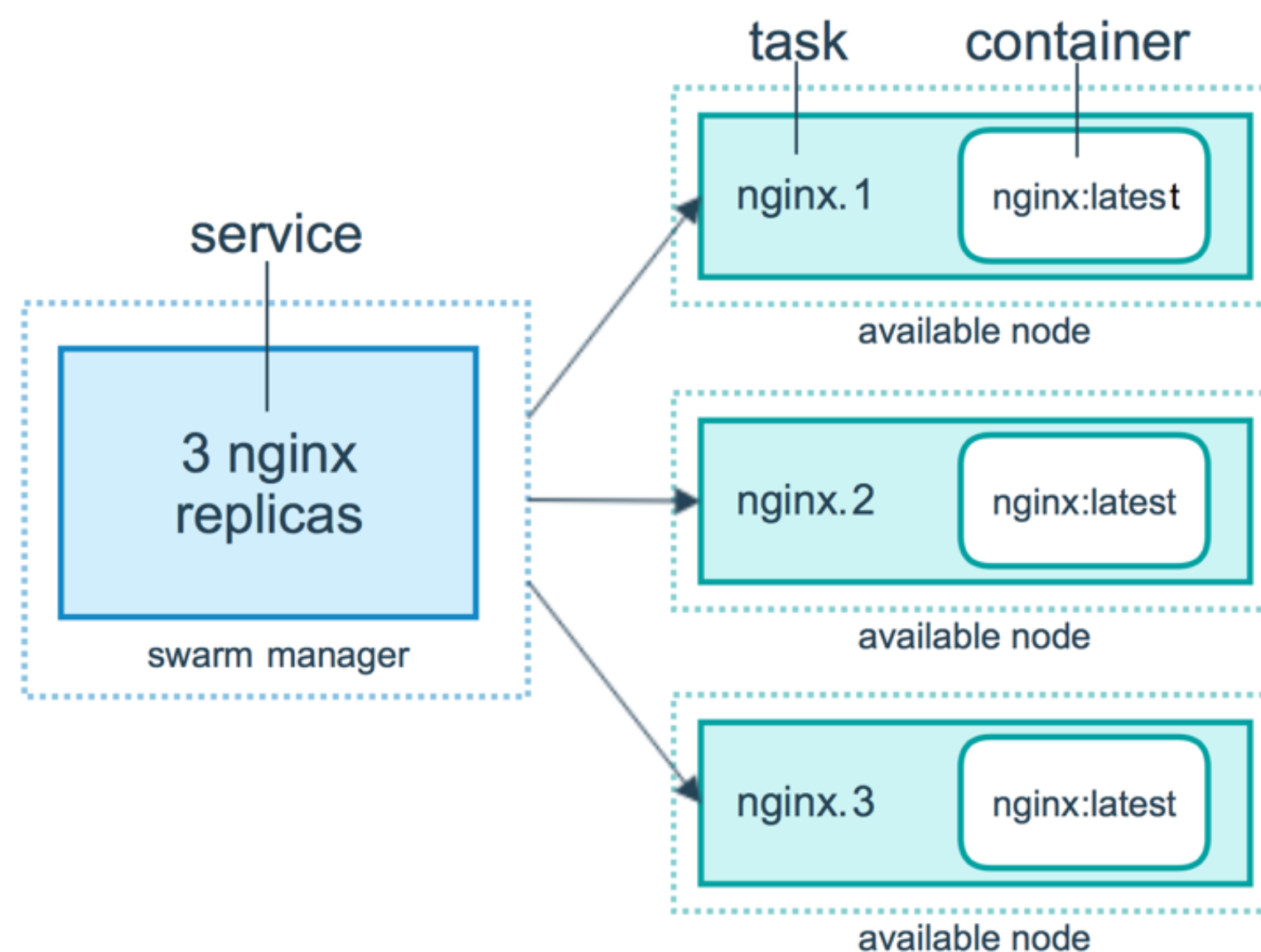
Swarm managers uses **ingress load balancing** to expose services to the external world.

Swarm managers can automatically assign the service a **PublishedPort**, if not specified.

In this case, the swarm managers assign the service a port in the 30000-32767 range.

External components can access the service on the PublishedPort of any node in the cluster —all nodes in the swarm route ingress connections to a running task instance.

Docker Swarm has an **internal DNS** that automatically assigns each service in the swarm a DNS entry. The swarm manager uses ingress load balancing to distribute requests among services within the cluster based on the DNS name of the service.



Kubernetes



Kubernetes (K8s) is an open-source platform for **automating deployment, scaling and operations related to containers** between clusters of nodes, providing a container-centric infrastructure.

With Kubernetes you can respond to customer requests quickly and efficiently:

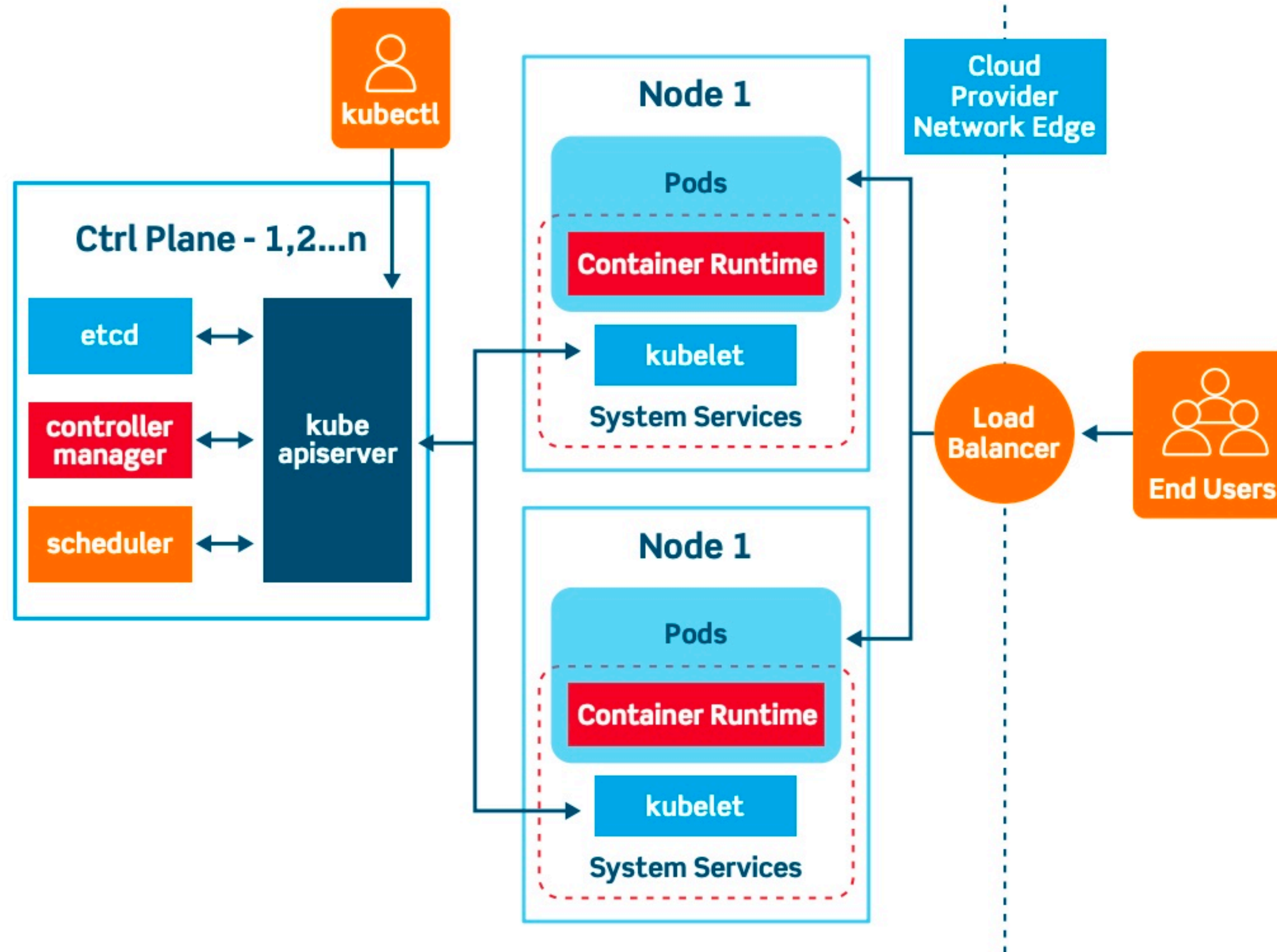
- Put applications into service quickly and predictably.
- Scale applications on the fly.
- Release new features invisibly to the customer.
- Optimize the use of hardware using only the necessary resources.

Kubernetes manages Docker containers and not machines!

Kubernetes is

- portable: manages public, private, hybrid and multi-cloud clouds;
- extensible: through modules, plugins, hooks and compositions;
- self-healing: through auto-placement, auto-restart, auto-replication, auto-scaling;
- 100% open-source, written in Go (initially only developed by Google).

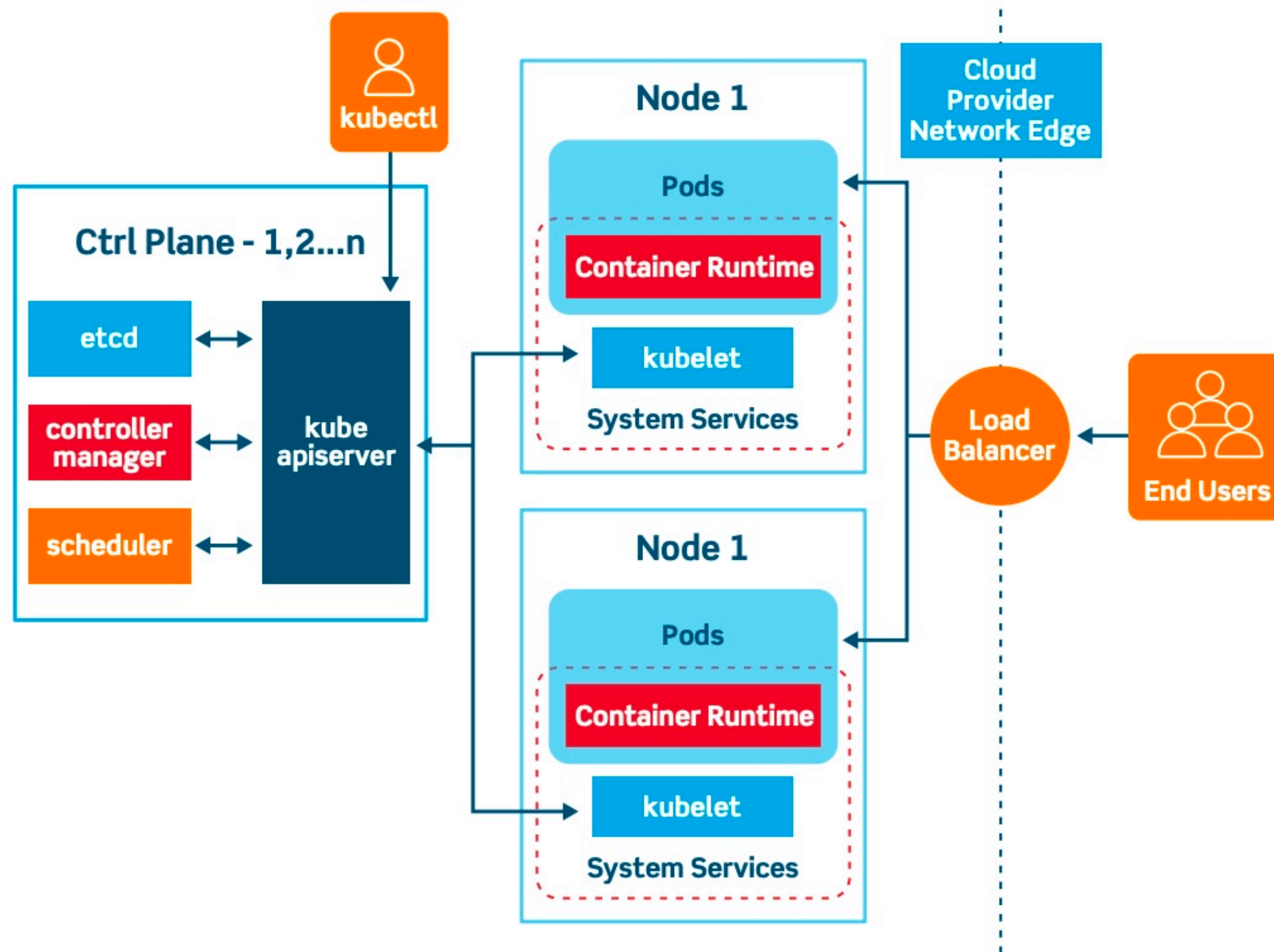
Kubernetes: Architecture



A Kubernetes cluster contains:

- a master node
- a series of agent nodes (kubelet)

Kubernetes: Architecture



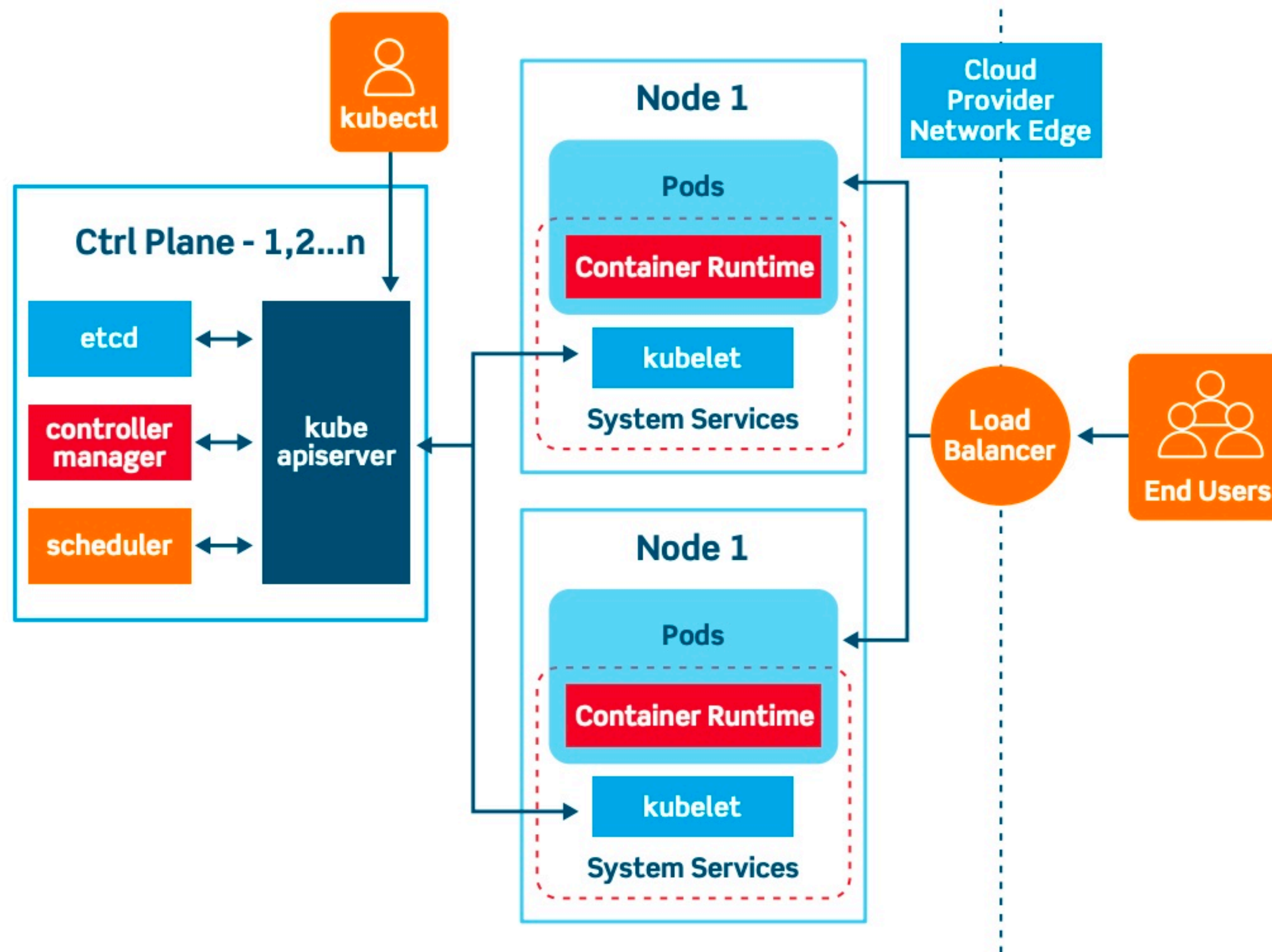
Kubelet

A kubelet has everything needed to run containers and be managed by the master node.

Each node contains a Docker instance and manages several pods (i.e., containers, images and volumes).

Each node has a network proxy and a load balancer.

Kubernetes: Architecture

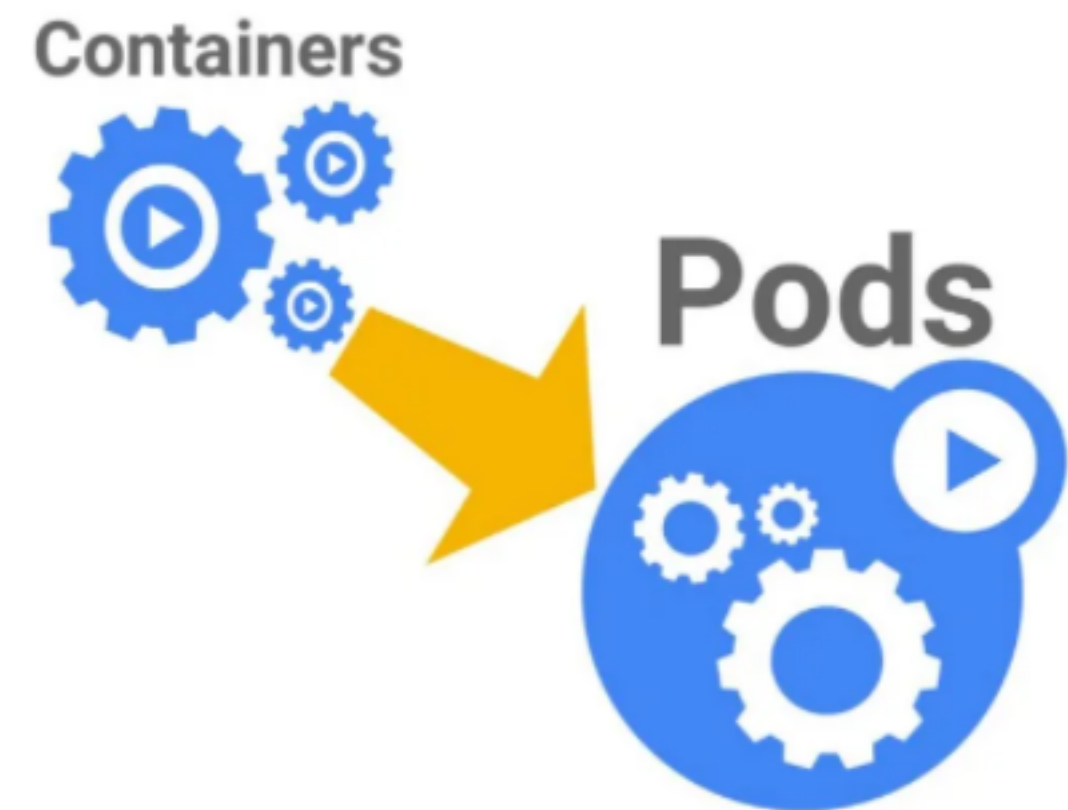


K8s Master

The master node manages the kubelets. It integrates several components:

- **kubectl**, manages kubelets.
- **etcd**, allows to save the configuration data of the nodes in a reliable way. Allows you to restore nodes in case of problems.
- **kube apiserver**, provides the kubernetes API in the form of a CRUD server consisting of plugins.
- **Scheduler**, assign pods to nodes through binding. It supports different types of clusters and can integrate information provided by the user
- **Controller manager**, all other cluster-level functions are managed by the controller manager.

Kubernetes: Basic Elements

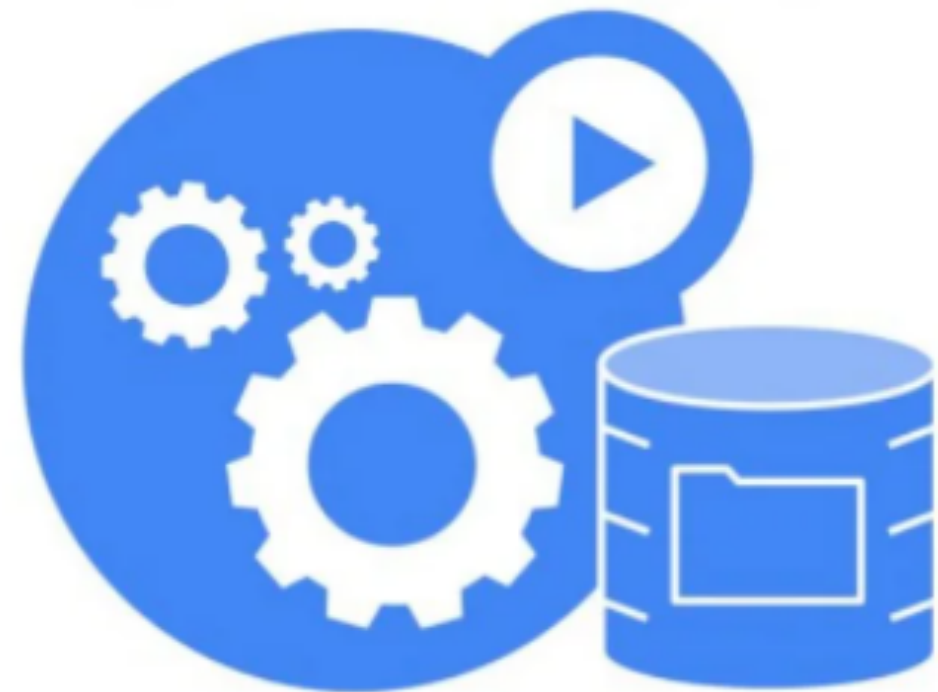


A pod is a group of co-located containers and volumes



A label is a key/value pair linked to a resource to identify it.

Volumes

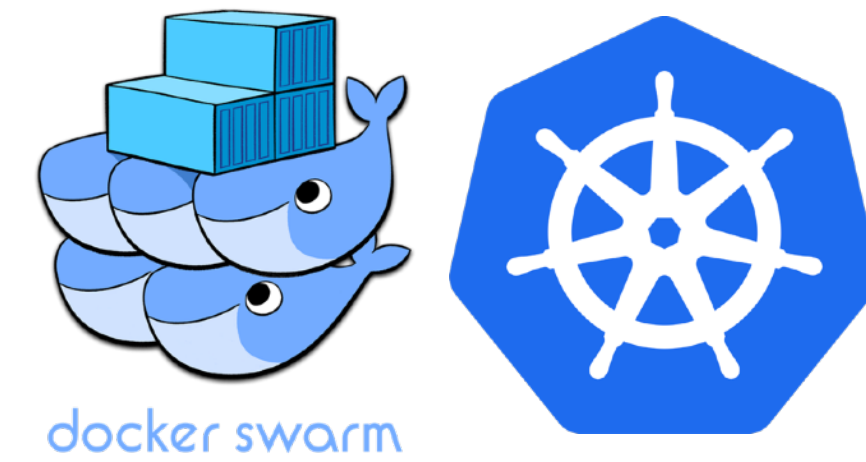


A volume is a data directory accessible by the container. Volume K8s are based on Docker volumes.



A replication controller ensures that a certain number of pod replicas are running at a certain time. Simplify scaling and manage pod creation in case of failure.

Docker Swarm vs Kubernetes



Docker Swarm

- + **Straightforward to install.**
- + **Lightweight.**
- + **Great learning curve.**
- + **Automated load balancing.**
- + **It works with the Docker CLI**
- + **No system changes if it is already running inside Docker.**
- + **Secure by default through TLS authentication and encryption.**
- + **Rolling updates.**
- **Tied to the Docker API, which limits its functionality.**
- **Automation capabilities not as robust as Kubernetes.**

Kubernetes

- + **Large open-source community, and Google backs it.**
- + **Supports every operating system.**
- + **Manage large architectures and complex workloads.**
- + **Self-healing capacity supporting automatic scaling.**
- + **Built-in monitoring and a wide range of available integrations.**
- + **Offered by Google, Azure, and AWS.**
- + **Rolling updates.**
- **Complex installation process**
- **Steep learning curve.**
- **Separate CLI tools.**

Kubernetes in Practice: Minikube

<https://minikube.sigs.k8s.io/docs/>

Installing Minikube

Select some sample applications, analyze them, and run them with Docker Compose!

<https://minikube.sigs.k8s.io/docs/start/>

Minikube 101: a Tutorial

https://minikube.sigs.k8s.io/docs/tutorials/kubernetes_101/